

# Cmod A7-35T RISC-V MCU

## Datasheet & Reference Manual

MicroBlaze-V soft MCU on the Digilent Cmod A7-35T — peripherals, pins, power, and boot guides

|                     |                                                |
|---------------------|------------------------------------------------|
| <b>Platform</b>     | Xilinx Artix-7 (xc7a35tcpg236-1) — Cmod A7-35T |
| <b>Processor</b>    | MicroBlaze RISC-V                              |
| <b>System Clock</b> | 100 MHz (12 MHz on-board oscillator × PLL)     |
| <b>Toolchain</b>    | Vivado & Vitis 2025.2                          |

Document DS-CMOD7-0001 · Revision A · July 2026

# Table of Contents

|       |                                           |    |
|-------|-------------------------------------------|----|
| 1     | Device Overview                           | 4  |
| 1.1   | Introduction                              | 4  |
| 1.2   | Features                                  | 4  |
| 1.3   | System Block Diagram                      | 4  |
| 1.4   | Reference Documents                       | 4  |
| 2     | System Architecture                       | 6  |
| 2.1   | MicroBlaze RISC-V Core                    | 6  |
| 2.2   | Bus Architecture                          | 6  |
| 2.3   | Interrupt Controller                      | 6  |
| 2.4   | Clocking                                  | 7  |
| 2.5   | Reset                                     | 7  |
| 2.6   | Debug Module                              | 7  |
| 2.7   | Complete Address Map                      | 7  |
| 3     | Memory Hierarchy                          | 8  |
| 3.1   | Memory Hierarchy Overview                 | 8  |
| 3.2   | Memory Map                                | 9  |
| 3.3   | Local Memory (ITCM / DTCM / Bootloader)   | 9  |
| 3.4   | External SRAM (Cellular RAM)              | 9  |
| 3.5   | QSPI Flash                                | 10 |
| 3.6   | Caches                                    | 10 |
| 3.7   | Measured Access Latencies                 | 10 |
| 3.8   | Memory Placement Guidance                 | 11 |
| 4     | Pinout                                    | 12 |
| 4.1   | DIP Connector Overview                    | 12 |
| 4.2   | Pin Map — Left Side (Pin 1–24)            | 13 |
| 4.3   | Pin Map — Right Side (Pin 25–48)          | 13 |
| 4.4   | On-Board I/O (No DIP Pin Exposure)        | 14 |
| 5     | Peripherals                               | 15 |
| 5.1   | GPIO                                      | 15 |
| 5.1.1 | On-Board GPIO                             | 15 |
| 5.1.2 | DIP Connector GPIO (4 Groups × 7-bit)     | 15 |
| 5.1.3 | External Interrupt Inputs (INT_0_3)       | 15 |
| 5.2   | Timers and PWM                            | 15 |
| 5.2.1 | System Timers                             | 15 |
| 5.2.2 | PWM Outputs                               | 15 |
| 5.3   | UART                                      | 16 |
| 5.3.1 | USB UART (uart_USB)                       | 16 |
| 5.3.2 | External UART (uart_1)                    | 16 |
| 5.4   | I2C Controller                            | 16 |
| 5.5   | SPI Master                                | 16 |
| 5.6   | Analog-to-Digital Converter (XADC)        | 16 |
| 5.6.1 | Analog Input Circuit                      | 17 |
| 6     | Electrical Characteristics and Power      | 18 |
| 6.1   | Electrical Characteristics                | 18 |
| 6.2   | Power Architecture                        | 18 |
| 6.3   | Power Input Options                       | 18 |
| 6.4   | Output Power Rails                        | 18 |
| 6.4.1 | Power-Up Sequence                         | 19 |
| 6.5   | VU Pin Behavior                           | 19 |
| 6.6   | Dual Power Source (USB + External)        | 19 |
| 6.7   | Quick Reference for External Power Design | 19 |
| 7     | JTAG Debug Mode                           | 20 |
| 7.1   | Prerequisites                             | 20 |
| 7.2   | Open the Workspace                        | 20 |
| 7.3   | Create a Platform                         | 21 |
| 7.3.1 | New Platform Component                    | 21 |
| 7.3.2 | Platform Name and Location                | 21 |
| 7.3.3 | Select the Hardware Design (XSA)          | 22 |
| 7.3.4 | Select Operating System and Processor     | 23 |
| 7.3.5 | Platform Summary                          | 23 |
| 7.4   | Create an Application                     | 24 |
| 7.4.1 | Create from Template                      | 24 |

|       |                                                    |    |
|-------|----------------------------------------------------|----|
| 7.4.2 | Application Name and Location .....                | 25 |
| 7.4.3 | Select the Platform .....                          | 25 |
| 7.4.4 | Select the Domain .....                            | 26 |
| 7.4.5 | Application Summary .....                          | 26 |
| 7.5   | Replace the Sources with the Course Template ..... | 27 |
| 7.5.1 | Delete the Generated Files .....                   | 27 |
| 7.5.2 | Import the Template Files .....                    | 28 |
| 7.5.3 | About the Linker Script .....                      | 29 |
| 7.6   | Set the Console UART (BSP) .....                   | 29 |
| 7.7   | Build .....                                        | 30 |
| 7.7.1 | Build the Platform .....                           | 30 |
| 7.7.2 | Build the Application .....                        | 30 |
| 7.8   | Debug over JTAG .....                              | 31 |
| 7.8.1 | Start a Debug Session .....                        | 31 |
| 7.8.2 | Open a Serial Terminal, Then Continue .....        | 32 |
| 7.8.3 | Breakpoints and Stepping .....                     | 33 |
| 7.9   | Inspection Tools .....                             | 34 |
| 7.9.1 | Memory Inspector .....                             | 34 |
| 7.9.2 | Register Inspector .....                           | 34 |
| 7.9.3 | Disassembly View .....                             | 35 |
| 7.10  | Troubleshooting .....                              | 35 |
| 8     | Standalone Boot .....                              | 38 |
| 8.1   | How Standalone Boot Works .....                    | 38 |
| 8.2   | Prerequisites .....                                | 38 |
| 8.3   | Create the Bootloader Component .....              | 38 |
| 8.3.1 | New Application from Template .....                | 38 |
| 8.3.2 | Name and Location .....                            | 39 |
| 8.3.3 | Select the Platform .....                          | 39 |
| 8.3.4 | Summary .....                                      | 40 |
| 8.4   | Configure the Bootloader .....                     | 40 |
| 8.4.1 | Set the Application Slot Address .....             | 40 |
| 8.4.2 | Disable Verbose Output .....                       | 41 |
| 8.5   | Build the Bootloader .....                         | 42 |
| 8.6   | Merge the Bootloader into the Hardware Image ..... | 43 |
| 8.7   | Prepare the Application Image .....                | 44 |
| 8.8   | Program the Flash .....                            | 45 |
| 8.8.1 | Program the Hardware Image .....                   | 46 |
| 8.8.2 | Program the Application .....                      | 47 |
| 8.9   | Boot and Verify .....                              | 48 |
| 8.10  | Updating the Application .....                     | 48 |
| 8.11  | Working with JTAG Debug Mode .....                 | 48 |
| 8.12  | Troubleshooting .....                              | 49 |

# 1 Device Overview

## 1.1 Introduction

The Cmod A7-35T RISC-V MCU is a soft microcontroller implemented on the Artix-7 FPGA of the Digilent Cmod A7-35T module. To the programmer it behaves like a commercial flash-based MCU: firmware is stored in the on-board QSPI flash and boots automatically in under a second at power-on, and the full AMD Vitis/JTAG development flow is available on the same board over a single USB cable, with no jumpers or mode switching.

The system is built around a MicroBlaze-V (RISC-V) processor with a three-level memory hierarchy and a class-based AXI peripheral address map. The memory hierarchy consists of tightly-coupled BRAM (ITCM/DTCM), cached external SRAM, and QSPI flash storage.

## 1.2 Features

- **CPU:** 32-bit RISC-V (RV32IM + bit-manipulation) at 100 MHz; 16 KB I-cache + 16 KB D-cache (write-through, 32-byte lines)
- **Memory:** 128 KB block RAM (32 KB bootloader + 32 KB ITCM + 64 KB DTCM, single-cycle); 512 KB SRAM for application code and data (cached); 4 MB QSPI flash (bitstream + application storage)
- **GPIO:** 28 bidirectional pins in four 7-bit groups, plus 4 dedicated external interrupt inputs
- **Timers:** 3 × 32-bit general-purpose timers with interrupts; 3 dedicated PWM output channels
- **Communication:** 2 × 16550 UART (USB + DIP header); I2C master (100 kHz, 50 ns glitch filter); SPI master (software-settable clock ≈ 1.6 – 25 MHz, 2 slave selects)
- **Analog:** 2 external ADC inputs (12-bit XADC, 0–3.3 V range) plus on-die temperature and supply monitors
- **I/O:** 48-pin DIP form factor, 3.3 V LVCMOS (not 5 V tolerant)
- **Boot:** standalone boot from flash (< 1 s) via the on-chip bootloader, or JTAG load/debug from Vitis; both coexist without reconfiguration
- **Debug:** JTAG breakpoints, single-step, register and memory inspection

## 1.3 System Block Diagram

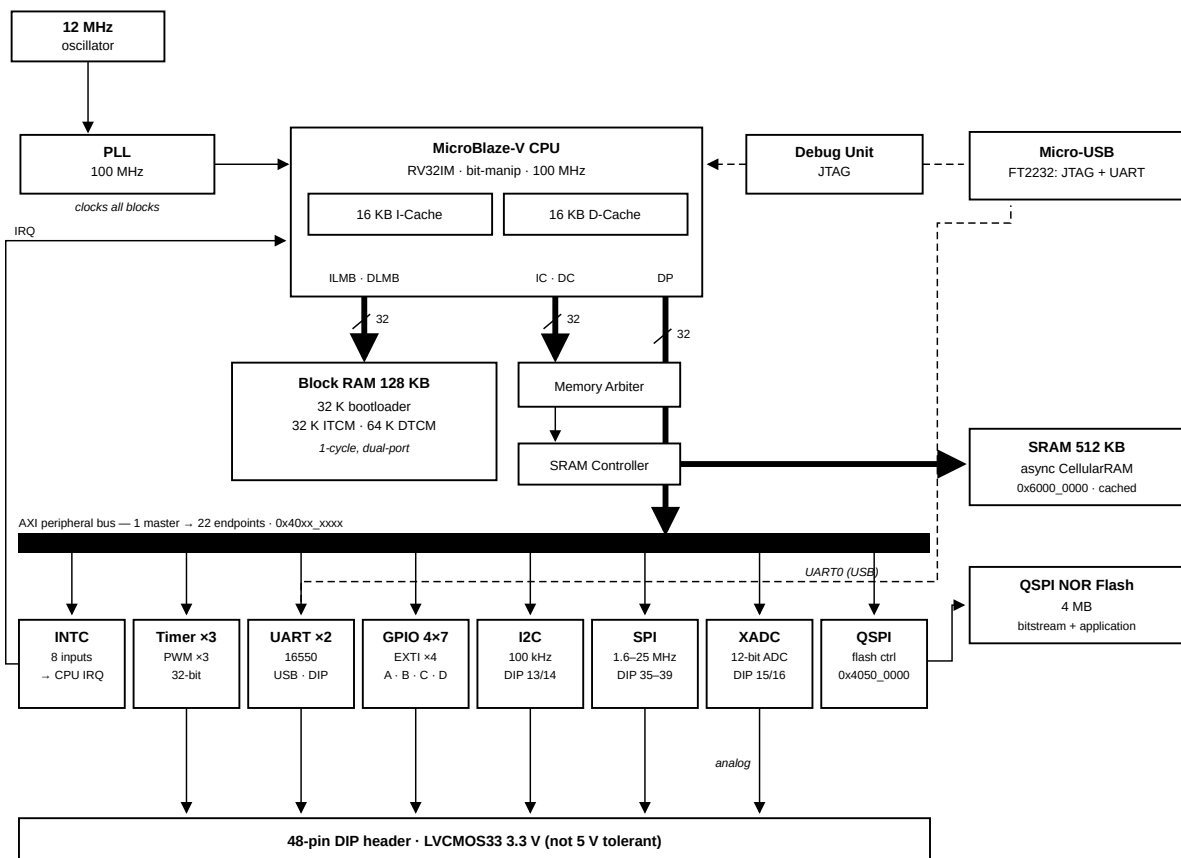


Figure 1: System Architecture

## 1.4 Reference Documents

- Digilent *Cmod A7 Reference Manual*: board schematics, connectors, and the power tree

- AMD/Xilinx *DS181, Artix-7 FPGAs Data Sheet*: absolute maximum ratings and DC/AC characteristics underlying Section 6
- AMD/Xilinx LogiCORE IP product guides (PG series): full register maps for the AXI peripherals (e.g. PG153 for the Quad SPI controller)
- The per-topic markdown under `docs/` in the course repository: the single source from which this datasheet is built

## 2 System Architecture

### 2.1 MicroBlaze RISC-V Core

| Item         | Value                                                                                                               |
|--------------|---------------------------------------------------------------------------------------------------------------------|
| Architecture | 32-bit RISC-V — RV32IM + bit-manipulation (hardware multiply/divide)                                                |
| Clock        | 100 MHz                                                                                                             |
| Buses        | LMB to local BRAM (1-cycle) · AXI to peripherals and external memory                                                |
| Cache        | 16 KB I-Cache + 16 KB D-Cache, write-through, 32 B lines — covers exactly the 512 KB SRAM (0x6000_0000–0x6007_FFFF) |
| Debug        | JTAG: breakpoints, register inspection, memory read/write                                                           |

**Description:** The main processor core executes user firmware and reaches every peripheral through the AXI interconnect. Only the SRAM range is cached: BRAM access is already single-cycle, and peripheral registers must never be cached.

### 2.2 Bus Architecture

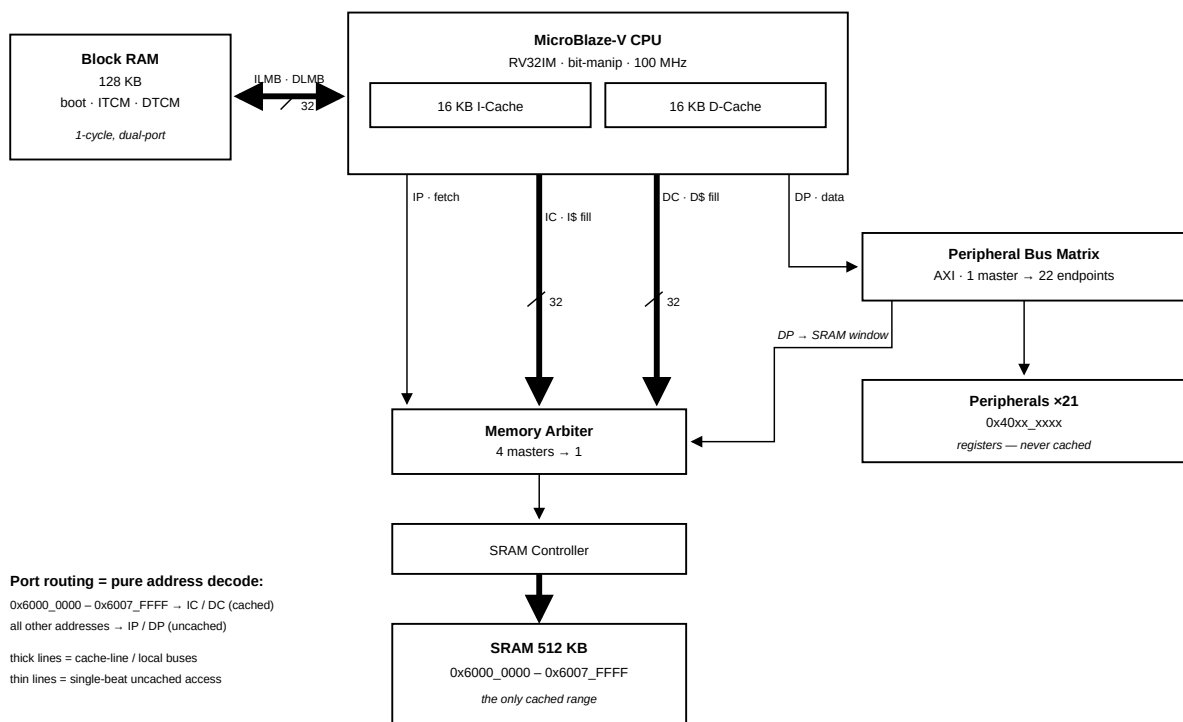


Figure 2: Bus & Cache Topology

**Description:** An AXI crossbar fans the processor's data port out to all peripheral endpoints. Routing is determined entirely by address decoding. The diagram above shows the path taken by each address range; a second, smaller interconnect merges the cache and debug masters in front of the SRAM controller.

### 2.3 Interrupt Controller

| Item              | Value       |
|-------------------|-------------|
| Base Address      | 0x4040_0000 |
| Interrupt Sources | 8           |

#### Interrupt Mapping:

| Channel | Source  | Description              |
|---------|---------|--------------------------|
| In0     | timer_0 | System timer 0 interrupt |
| In1     | timer_1 | System timer 1 interrupt |
| In2     | timer_2 | System timer 2 interrupt |

| Channel | Source   | Description                     |
|---------|----------|---------------------------------|
| In3     | uart_1   | External UART interrupt         |
| In4     | uart_USB | USB UART interrupt              |
| In5     | INT_0_3  | External GPIO interrupt (4-bit) |
| In6     | i2c_0    | I2C controller interrupt        |
| In7     | spi_0    | External SPI master interrupt   |

## 2.4 Clocking

**Description:** This block multiplies the 12 MHz on-board oscillator up to the 100 MHz system clock with an MMCM/PLL. Its locked signal indicates clock stability and gates the release of system reset.

## 2.5 Reset

**Description:** This block generates synchronized reset signals for the CPU, bus fabric and peripherals, releasing them only after the clock is stable. The external reset source is the on-board button BTN0 (A18, active-high).

## 2.6 Debug Module

**Description:** The MicroBlaze Debug Module provides JTAG debug access for breakpoints, register inspection, and memory read/write. It can also trigger a system reset from the debugger (used by xsdb / Vitis debug sessions).

## 2.7 Complete Address Map

Peripheral addresses follow a class-based convention:  $0x40[C]x\_xxxx$ , where  $C$  is the peripheral class (1 MB per class, 64 KB per instance). The device type is identifiable directly from the address: class 0 = GPIO, 1 = Timer, 2 = PWM, 3 = UART, 4 = INTC, 5 = QSPI control, 6 = XADC, 7 = I2C, 8 = SPI, 9 = SPI clock control.

| Base Address | Range       | Peripheral                                 | Type          | Category      |
|--------------|-------------|--------------------------------------------|---------------|---------------|
| 0x0000_0000  | 128K / 128K | Local Memory (BRAM)                        | BRAM          | Memory        |
| 0x4000_0000  | 64K         | board_led_2bits                            | axi_gpio      | GPIO          |
| 0x4001_0000  | 64K         | board_button                               | axi_gpio      | GPIO          |
| 0x4002_0000  | 64K         | board_rgb                                  | axi_gpio      | GPIO          |
| 0x4003_0000  | 64K         | gpio_A_0_6                                 | axi_gpio      | GPIO          |
| 0x4004_0000  | 64K         | gpio_B_0_6                                 | axi_gpio      | GPIO          |
| 0x4005_0000  | 64K         | gpio_C_0_6                                 | axi_gpio      | GPIO          |
| 0x4006_0000  | 64K         | gpio_D_0_6                                 | axi_gpio      | GPIO          |
| 0x4007_0000  | 64K         | INT_0_3                                    | axi_gpio      | Interrupt     |
| 0x4010_0000  | 64K         | timer_0                                    | axi_timer     | Timer         |
| 0x4011_0000  | 64K         | timer_1                                    | axi_timer     | Timer         |
| 0x4012_0000  | 64K         | timer_2                                    | axi_timer     | Timer         |
| 0x4020_0000  | 64K         | PWM_0                                      | axi_timer     | PWM           |
| 0x4021_0000  | 64K         | PWM_1                                      | axi_timer     | PWM           |
| 0x4022_0000  | 64K         | PWM_2                                      | axi_timer     | PWM           |
| 0x4030_0000  | 64K         | uart_USB                                   | axi_uart16550 | Communication |
| 0x4031_0000  | 64K         | uart_1                                     | axi_uart16550 | Communication |
| 0x4040_0000  | 64K         | axi_intc                                   | axi_intc      | System        |
| 0x4050_0000  | 64K         | axi_quad_spi_0 (QSPI flash controller)     | axi_quad_spi  | Memory        |
| 0x4060_0000  | 64K         | xadc_wiz_0                                 | xadc_wiz      | ADC           |
| 0x4070_0000  | 64K         | i2c_0 (DIP 13/14)                          | axi_iic       | Communication |
| 0x4080_0000  | 64K         | spi_0 (external master, DIP 35–39)         | axi_quad_spi  | Communication |
| 0x4090_0000  | 64K         | spi_0_clk (SPI clock control)              | clock control | Communication |
| 0x6000_0000  | 512K        | axi_emc_0 (exact physical fit, I/D-cached) | axi_emc       | Memory        |

## 3 Memory Hierarchy

### 3.1 Memory Hierarchy Overview

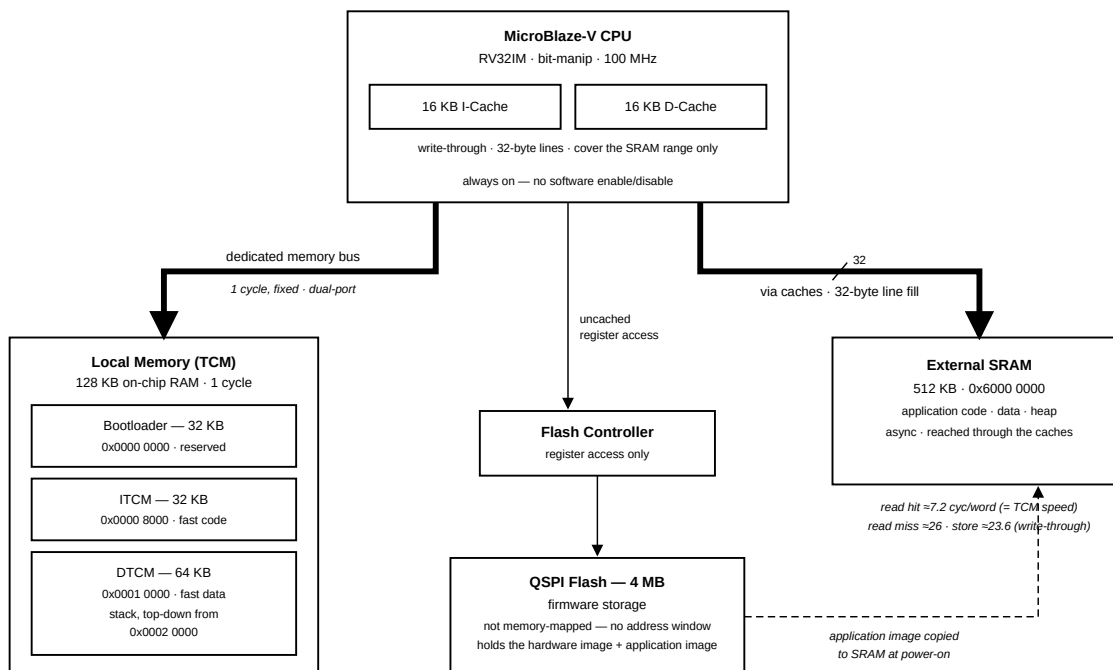


Figure 3: Memory Hierarchy

| Level | Memory                            | Size          | Access                   | Role                                |
|-------|-----------------------------------|---------------|--------------------------|-------------------------------------|
| 1     | ITCM + DTCM (tightly-coupled RAM) | 32 KB + 64 KB | 1 cycle, fixed           | interrupt handlers, stack, hot data |
| 2     | I-Cache + D-Cache                 | 16 KB + 16 KB | adds ≈ 0 cycles on hit   | transparent acceleration of SRAM    |
| 3     | External SRAM                     | 512 KB        | ≈ 26 cycles/word on miss | application code, data, heap        |
| —     | QSPI flash                        | 4 MB          | not directly addressable | firmware storage, read at boot      |

The tightly-coupled memories reside on a dedicated dual-port memory bus with guaranteed single-cycle access; they are never cached because they are already as fast as a cache hit. The external SRAM is the application's main memory; it is reached through the caches, so frequently accessed code and data run at tightly-coupled speed while the total space is 512 KB. The QSPI flash has no CPU address window: firmware is stored in flash and executed from SRAM, and the bootloader performs the copy at every power-on.

## 3.2 Memory Map

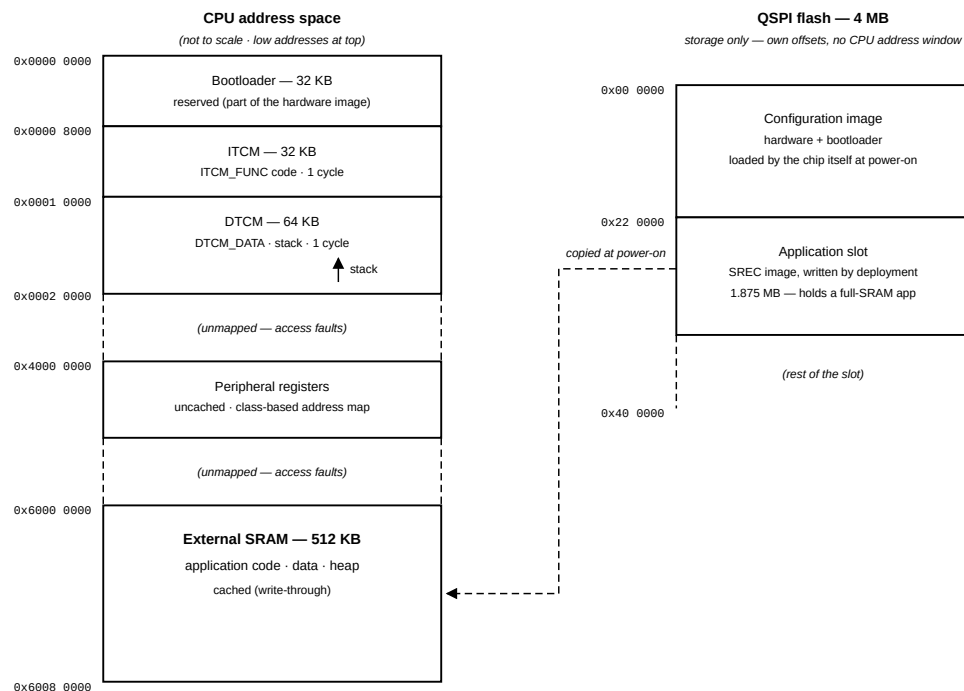


Figure 4: Memory Map

| Address range             | Size   | Region                                              | Access   |
|---------------------------|--------|-----------------------------------------------------|----------|
| 0x0000_0000 — 0x0000_7FFF | 32 KB  | Bootloader (reserved)                               | 1 cycle  |
| 0x0000_8000 — 0x0000_FFFF | 32 KB  | ITCM — application fast code                        | 1 cycle  |
| 0x0001_0000 — 0x0001_FFFF | 64 KB  | DTCM — fast data; stack grows down from 0x0002_0000 | 1 cycle  |
| 0x4000_0000 — 0x40FF_FFFF | —      | Peripheral registers                                | uncached |
| 0x6000_0000 — 0x6007_FFFF | 512 KB | External SRAM — application code / data / heap      | cached   |

Address decoding is exact: an access outside these ranges raises a bus error instead of aliasing onto real memory, so an invalid pointer access is detected rather than silently corrupting data. The QSPI flash does not appear in this map; it is accessed only through the flash controller's registers. Peripheral registers and base addresses are listed in Section 2.7 and detailed in Section 5.

## 3.3 Local Memory (ITCM / DTCM / Bootloader)

| Item    | Value                                                               |
|---------|---------------------------------------------------------------------|
| Address | 0x0000_0000 — 0x0001_FFFF (128 KB)                                  |
| Access  | 1 cycle, dual-port — instruction and data sides read simultaneously |
| Layout  | 32 KB bootloader + 32 KB ITCM + 64 KB DTCM                          |

**Description:** Instruction and data local memory is implemented with FPGA Block RAM and functions as this MCU's tightly-coupled memory (TCM): the ILMB/DLMB ports serve the same role as ITCM/DTCM on other MCU cores (e.g. Cortex-M7), giving 1-cycle access outside the cache path. The layout is as follows: 0x0000–0x7FFF (32 KB) holds the bootloader (restored from flash at every configuration); 0x8000–0xFFFF (32 KB) is the application's ITCM for interrupt handlers and timing-critical code (ITCM\_FUNC, copied out of the SRAM image by `mcu_init()` at startup); 0x10000–0x1FFFF (64 KB) is the DTCM, holding the stack (top-down from 0x20000) and DTCM\_DATA fast data.

## 3.4 External SRAM (Cellular RAM)

| Item         | Value       |
|--------------|-------------|
| Base Address | 0x6000_0000 |

| Item   | Value                                                     |
|--------|-----------------------------------------------------------|
| Size   | 512 KB — 0x6000_0000 – 0x6007_FFFF, an exact physical fit |
| Memory | On-board asynchronous SRAM (Cellular RAM)                 |
| Cache  | Fully covered by the I-Cache and D-Cache                  |

**Description:** This external memory controller interfaces to the on-board 512 KB SRAM, the main application memory: standalone boot copies the program here and executes it. Raw access latency is higher than Block RAM, but with both caches covering this range, frequently accessed code and data run at close to BRAM speed.

### 3.5 QSPI Flash

| Item         | Value                                                                 |
|--------------|-----------------------------------------------------------------------|
| Base Address | 0x4050_0000                                                           |
| Flash Device | On-board 4 MB QSPI NOR flash (Macronix; Micron on older boards)       |
| Mode         | CPU-programmable register mode — flash contents are not memory-mapped |
| FIFO         | 256 B TX/RX — one full flash page (256 B) per transfer                |

**Description:** This controller manages the on-board Quad-SPI NOR Flash. The full register set (control, status, TX/RX FIFO, slave select) is exposed at 0x4050\_0000, so the CPU can issue any SPI command: read (0x0B), Write Enable (0x06), Sector/Block Erase (0x20/0xD8), Page Program (0x02), and status poll (0x05). The flash therefore remains fully CPU-programmable at runtime (the Vitis xSpi driver wraps the register protocol).

Flash contents are not memory-mapped: there is no XIP window, and code cannot execute from flash directly. The standalone-boot design instead copies the application from flash into SRAM at power-on (see Section 8).

**Design note:** A read-only memory-mapped XIP window and CPU-programmable register mode are mutually exclusive in this controller. This project uses register mode: keeping the flash CPU-programmable at runtime was judged more valuable for the course than execute-in-place, and execution is served by the 512 KB SRAM and the I-cache instead.

### 3.6 Caches

| Item              | Value                                            |
|-------------------|--------------------------------------------------|
| Instruction cache | 16 KB, 32-byte lines                             |
| Data cache        | 16 KB, 32-byte lines                             |
| Write policy      | write-through — every store is forwarded to SRAM |
| Cached range      | exactly the SRAM: 0x6000_0000 – 0x6007_FFFF      |
| Software control  | none — always on for the cached range            |

Only the SRAM range is cached. The tightly-coupled memories are already single-cycle, and peripheral registers must observe every access, so neither is ever cached. No cache enable, disable, or flush operations are required in normal application code.

The write-through policy has two practical consequences:

- Reads benefit fully from the cache: a hit has the same latency as a tightly-coupled RAM access.
- Writes always incur the SRAM latency, even on a cache hit. Frequently written data (counters, buffers filled in interrupt handlers) should therefore be placed in the DTCM.

### 3.7 Measured Access Latencies

Latencies were measured on the board at 100 MHz with an optimization-enabled word-copy loop. Values are cycles per 32-bit word and include the loop's own  $\approx 7$ -cycle overhead; the difference between rows is the memory's contribution.

| Access                       | Cycles/word    | Notes                                           |
|------------------------------|----------------|-------------------------------------------------|
| Tightly-coupled RAM read     | $\approx 7.2$  | equals the bare loop; the memory adds no cycles |
| SRAM read, cache <b>hit</b>  | $\approx 7.2$  | a hit is as fast as tightly-coupled RAM         |
| SRAM read, cache <b>miss</b> | $\approx 26$   | $\approx 151$ cycles to fill one 32-byte line   |
| SRAM write (write-through)   | $\approx 23.6$ | every store pays the SRAM latency               |

**Note:** Measure with compiler optimization enabled. An unoptimized build inflates the loop overhead to the point where cached and uncached accesses appear almost identical; the memory effects are hidden by the extra instructions.

### 3.8 Memory Placement Guidance

| What                                          | Put it in  | How (course template)                        |
|-----------------------------------------------|------------|----------------------------------------------|
| Interrupt handlers, timing-critical loops     | ITCM       | tag the function <code>ITCM_FUNC</code>      |
| Frequently-written data, DMA-like buffers     | DTCM       | tag the variable <code>DTCM_DATA</code>      |
| Stack                                         | DTCM       | placed there by the template's linker script |
| Everything else — code, constants, data, heap | SRAM       | default                                      |
| Firmware image                                | QSPI flash | written by the deployment step               |

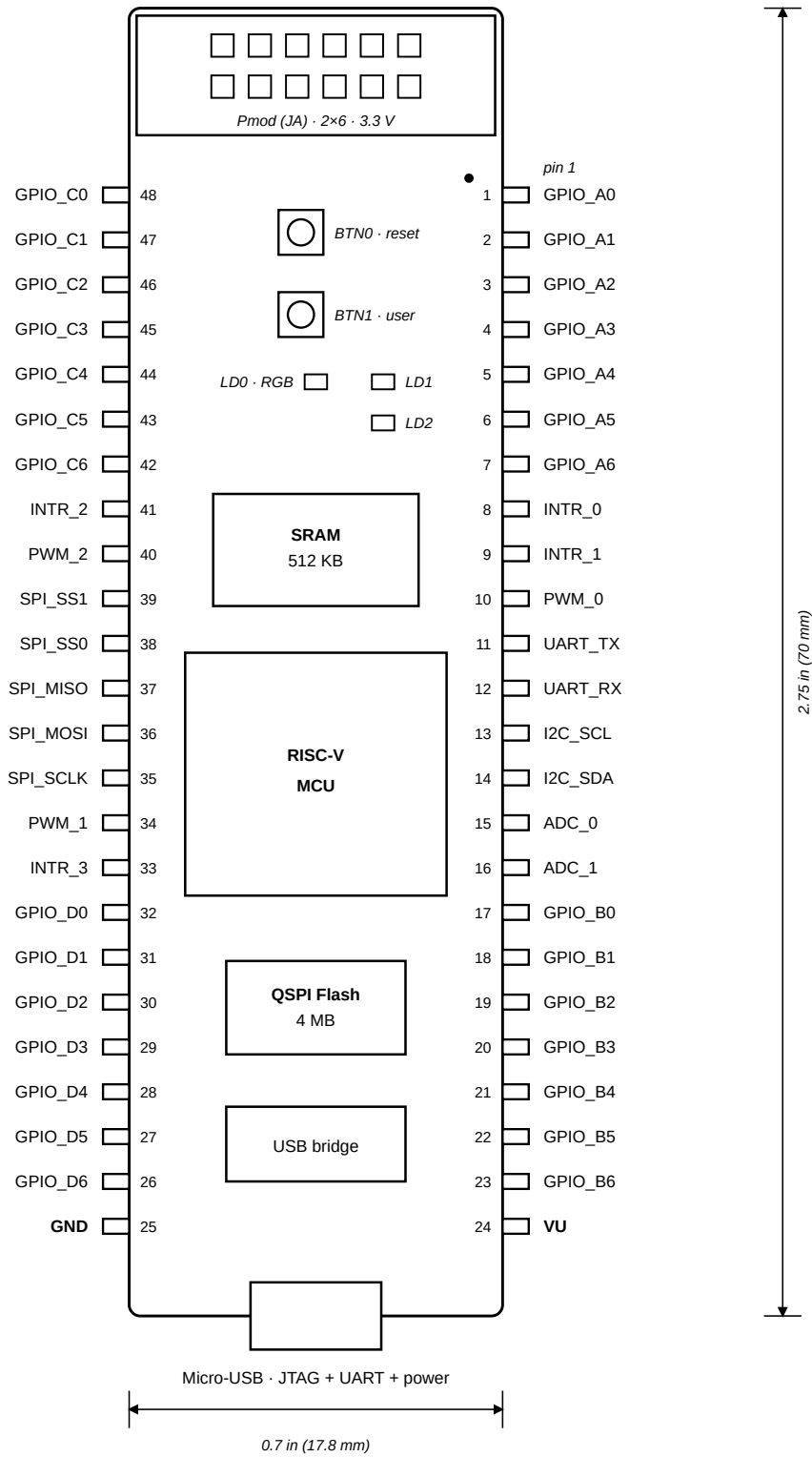
As a general rule, place code and data in SRAM by default and move them to the TCMs when predictable timing is required. A function in SRAM usually runs at cache speed, but its worst case (a cold cache line) takes three to four times longer; for an interrupt handler this spread appears directly as latency jitter. In the tightly-coupled memories the worst-case latency equals the best-case latency.

At power-on the bootloader copies the application image from flash into SRAM (see Section 8); the template's `mcu_init()` then copies `ITCM_FUNC` code out of the SRAM image into the ITCM and zeroes the DTCM. For this reason the template requires `mcu_init()`; to remain the first line of `main()`: contents placed in the tightly-coupled memories are not valid before it runs. The memory tour printed at boot lists each region's run-time addresses and serves as a check that the layout is in effect.

# 4 Pinout

## 4.1 DIP Connector Overview

The Cmod A7-35T has a 48-pin DIP connector (J1). All digital I/O pins operate at LVCMOS33 (3.3 V logic level) with no series resistors.



top view · Pmod end up · LVCMOS33 (pins 15/16 analog via divider)

Figure 5: Cmod A7-35T DIP pinout (top view)

| Category                  | Count     |
|---------------------------|-----------|
| GPIO (4 groups × 7-bit)   | 28        |
| External Interrupts       | 4         |
| PWM Outputs               | 3         |
| UART 1 (TX + RX)          | 2         |
| I2C (SCL + SDA)           | 2         |
| SPI (SCLK/MOSI/MISO/SS×2) | 5         |
| Analog Inputs (XADC)      | 2         |
| Power (VU + GND)          | 2         |
| <b>Total</b>              | <b>48</b> |

Peripheral registers and base addresses are listed in Section 2.7 and detailed in Section 5.

## 4.2 Pin Map — Left Side (Pin 1–24)

| DIP Pin | Signal Name | FPGA Pin | Direction        | Function                                        |
|---------|-------------|----------|------------------|-------------------------------------------------|
| 1       | GPIO_A0     | M3       | I/O              | General purpose GPIO, Group A bit 0             |
| 2       | GPIO_A1     | L3       | I/O              | General purpose GPIO, Group A bit 1             |
| 3       | GPIO_A2     | A16      | I/O              | General purpose GPIO, Group A bit 2             |
| 4       | GPIO_A3     | K3       | I/O              | General purpose GPIO, Group A bit 3             |
| 5       | GPIO_A4     | C15      | I/O              | General purpose GPIO, Group A bit 4             |
| 6       | GPIO_A5     | H1       | I/O              | General purpose GPIO, Group A bit 5             |
| 7       | GPIO_A6     | A15      | I/O              | General purpose GPIO, Group A bit 6             |
| 8       | INTR_0      | B15      | Input            | External interrupt input 0                      |
| 9       | INTR_1      | A14      | Input            | External interrupt input 1                      |
| 10      | PWM_0       | J3       | Output           | PWM output channel 0                            |
| 11      | UART_TX     | J1       | Output           | UART 1 transmit                                 |
| 12      | UART_RX     | K2       | Input            | UART 1 receive                                  |
| 13      | I2C_SCL     | L1       | I/O (open-drain) | I2C clock (external 4.7 kΩ pull-up recommended) |
| 14      | I2C_SDA     | L2       | I/O (open-drain) | I2C data (external 4.7 kΩ pull-up recommended)  |
| 15      | ADC_0       | G3 / G2  | Analog           | XADC VAUX4 (ain_p[15] / ain_n[15])              |
| 16      | ADC_1       | H2 / J2  | Analog           | XADC VAUX12 (ain_p[16] / ain_n[16])             |
| 17      | GPIO_B0     | M1       | I/O              | General purpose GPIO, Group B bit 0             |
| 18      | GPIO_B1     | N3       | I/O              | General purpose GPIO, Group B bit 1             |
| 19      | GPIO_B2     | P3       | I/O              | General purpose GPIO, Group B bit 2             |
| 20      | GPIO_B3     | M2       | I/O              | General purpose GPIO, Group B bit 3             |
| 21      | GPIO_B4     | N1       | I/O              | General purpose GPIO, Group B bit 4             |
| 22      | GPIO_B5     | N2       | I/O              | General purpose GPIO, Group B bit 5             |
| 23      | GPIO_B6     | P1       | I/O              | General purpose GPIO, Group B bit 6             |
| 24      | VU          | —        | Power            | Power input (ext) / Power output (USB)          |

## 4.3 Pin Map — Right Side (Pin 25–48)

| DIP Pin | Signal Name | FPGA Pin | Direction | Function                            |
|---------|-------------|----------|-----------|-------------------------------------|
| 25      | GND         | —        | Power     | Ground reference                    |
| 26      | GPIO_D6     | R3       | I/O       | General purpose GPIO, Group D bit 6 |
| 27      | GPIO_D5     | T3       | I/O       | General purpose GPIO, Group D bit 5 |
| 28      | GPIO_D4     | R2       | I/O       | General purpose GPIO, Group D bit 4 |
| 29      | GPIO_D3     | T1       | I/O       | General purpose GPIO, Group D bit 3 |
| 30      | GPIO_D2     | T2       | I/O       | General purpose GPIO, Group D bit 2 |
| 31      | GPIO_D1     | U1       | I/O       | General purpose GPIO, Group D bit 1 |
| 32      | GPIO_D0     | W2       | I/O       | General purpose GPIO, Group D bit 0 |

| DIP Pin | Signal Name | FPGA Pin | Direction | Function                                    |
|---------|-------------|----------|-----------|---------------------------------------------|
| 33      | INTR_3      | V2       | Input     | External interrupt input 3                  |
| 34      | PWM_1       | W3       | Output    | PWM output channel 1                        |
| 35      | SPI_SCLK    | V3       | Output    | SPI clock (software-settable, up to 25 MHz) |
| 36      | SPI_MOSI    | W5       | Output    | SPI master-out slave-in                     |
| 37      | SPI_MISO    | V4       | Input     | SPI master-in slave-out                     |
| 38      | SPI_SS0     | U4       | Output    | SPI slave select 0 (active low)             |
| 39      | SPI_SS1     | V5       | Output    | SPI slave select 1 (active low)             |
| 40      | PWM_2       | W4       | Output    | PWM output channel 2                        |
| 41      | INTR_2      | U5       | Input     | External interrupt input 2                  |
| 42      | GPIO_C6     | U2       | I/O       | General purpose GPIO, Group C bit 6         |
| 43      | GPIO_C5     | W6       | I/O       | General purpose GPIO, Group C bit 5         |
| 44      | GPIO_C4     | U3       | I/O       | General purpose GPIO, Group C bit 4         |
| 45      | GPIO_C3     | U7       | I/O       | General purpose GPIO, Group C bit 3         |
| 46      | GPIO_C2     | W7       | I/O       | General purpose GPIO, Group C bit 2         |
| 47      | GPIO_C1     | U8       | I/O       | General purpose GPIO, Group C bit 1         |
| 48      | GPIO_C0     | V8       | I/O       | General purpose GPIO, Group C bit 0         |

#### 4.4 On-Board I/O (No DIP Pin Exposure)

| Function            | FPGA Pins     |
|---------------------|---------------|
| LEDs (2)            | A17, C16      |
| User button (BTN1)  | B18           |
| Reset button (BTN0) | A18           |
| RGB LED             | B17, B16, C17 |

## 5 Peripherals

All peripheral registers are memory-mapped and reached through the AXI data port; base addresses follow the class-based scheme summarised in Section 2.7. Pin locations for every signal are listed in the pin maps of Section 4. The QSPI flash controller is described with the memory system in Section 3.5.

### 5.1 GPIO

All GPIO groups are memory-mapped ports with per-bit direction control: the TRI register sets each pin's direction, and the DATA register reads or writes the pin. The Vitis driver is `XGpio`.

#### 5.1.1 On-Board GPIO

| Instance        | Base Address | Width | Direction | Connection    | Description                                                 |
|-----------------|--------------|-------|-----------|---------------|-------------------------------------------------------------|
| board_led_2bits | 0x4000_0000  | 2     | Output    | A17, C16      | On-board LEDs × 2                                           |
| board_button    | 0x4001_0000  | 1     | Input     | B18           | On-board user button (BTN1)                                 |
| board_rgb       | 0x4002_0000  | 3     | Output    | B17, B16, C17 | On-board RGB LED (bit 0 = Blue, bit 1 = Green, bit 2 = Red) |

#### 5.1.2 DIP Connector GPIO (4 Groups × 7-bit)

| Instance   | Base Address | Width | DIP Pins  | Description                     |
|------------|--------------|-------|-----------|---------------------------------|
| gpio_A_0_6 | 0x4003_0000  | 7     | Pin 1–7   | GPIO Group A, bidirectional I/O |
| gpio_B_0_6 | 0x4004_0000  | 7     | Pin 17–23 | GPIO Group B, bidirectional I/O |
| gpio_C_0_6 | 0x4005_0000  | 7     | Pin 42–48 | GPIO Group C, bidirectional I/O |
| gpio_D_0_6 | 0x4006_0000  | 7     | Pin 26–32 | GPIO Group D, bidirectional I/O |

**Description:** Each group is a 7-bit bidirectional port; every bit can be an input or an output independently.

#### 5.1.3 External Interrupt Inputs (INT\_0\_3)

| Item         | Value                                                            |
|--------------|------------------------------------------------------------------|
| Base Address | 0x4007_0000                                                      |
| Width        | 4-bit, input-only                                                |
| DIP Pins     | Pin 8 (INTR_0), Pin 9 (INTR_1), Pin 41 (INTR_2), Pin 33 (INTR_3) |
| Interrupt    | INTC In5                                                         |

**Description:** Four external interrupt inputs are grouped into a single GPIO instance with interrupt generation enabled. A change on any of the four pins can raise INTC In5.

### 5.2 Timers and PWM

The device provides six 32-bit AXI timer instances (Vitis driver: `XTmrCtr`): three serve as general-purpose timers with interrupts, and three have their outputs routed to pins as PWM channels.

#### 5.2.1 System Timers

| Instance | Base Address | Interrupt | Description                  |
|----------|--------------|-----------|------------------------------|
| timer_0  | 0x4010_0000  | INTC In0  | General-purpose system timer |
| timer_1  | 0x4011_0000  | INTC In1  | General-purpose timer        |
| timer_2  | 0x4012_0000  | INTC In2  | General-purpose timer        |

#### 5.2.2 PWM Outputs

| Instance | Base Address | Output Pin | DIP Pin | Description   |
|----------|--------------|------------|---------|---------------|
| PWM_0    | 0x4020_0000  | J3         | Pin 10  | PWM Channel 0 |
| PWM_1    | 0x4021_0000  | W3         | Pin 34  | PWM Channel 1 |
| PWM_2    | 0x4022_0000  | W4         | Pin 40  | PWM Channel 2 |

**Description:** These timer instances are configured in PWM mode to generate square-wave outputs, suitable for applications such as LED dimming, motor speed control, and buzzer tone generation. Frequency and duty cycle are configured through the Timer Load Registers and the PWM enable bit.

## 5.3 UART

### 5.3.1 USB UART (uart\_USB)

| Item         | Value                                                  |
|--------------|--------------------------------------------------------|
| Base Address | 0x4030_0000                                            |
| TX / RX Pins | J18 / J17 — routed to the on-board Micro-USB connector |
| Interrupt    | INTC In4                                               |

**Description:** A 16550-compatible UART provides host PC communication through the on-board Micro-USB connector. The baud rate is software-configured via the divisor latch: at a 100 MHz system clock, divisor 54 (0x36) gives 115200 baud. This UART is typically mapped as STDIN/STDOUT.

### 5.3.2 External UART (uart\_1)

| Item         | Value                     |
|--------------|---------------------------|
| Base Address | 0x4031_0000               |
| TX / RX Pins | J1 (DIP 11) / K2 (DIP 12) |
| Interrupt    | INTC In3                  |

**Description:** A second 16550 UART is exposed on the DIP connector for communication with external devices (e.g., sensor modules, Bluetooth modules).

## 5.4 I2C Controller

| Item                | Value                                                                                      |
|---------------------|--------------------------------------------------------------------------------------------|
| Base Address        | 0x4070_0000                                                                                |
| SCL Frequency       | 100 kHz (standard mode)                                                                    |
| Input Glitch Filter | 50 ns on SCL and SDA — per the I2C tSP spike-suppression spec                              |
| SCL / SDA Pins      | DIP Pin 13 (L1) / Pin 14 (L2)                                                              |
| Pull-ups            | Weak FPGA internal pull-ups enabled; external 4.7 kΩ to 3.3 V recommended for real devices |
| Interrupt           | INTC In6                                                                                   |

**Description:** An AXI IIC master controls external I2C devices (sensors, EEPROMs, OLED displays). The bus uses open-drain signaling: any device may only pull the line low, and the pull-up resistor returns it high. Devices are addressed by their 7-bit I2C address. Interrupt routing is listed in Section 2.3. Use the Vitis `xiic` driver.

## 5.5 SPI Master

| Item          | Value                                                                                                                  |
|---------------|------------------------------------------------------------------------------------------------------------------------|
| Base Address  | 0x4080_0000                                                                                                            |
| SCLK          | Software-programmable, ≈1.6 – 25 MHz (6.25 MHz at reset)                                                               |
| Clock Control | 0x4090_0000 — runtime clock setting; see <code>showcase/src/spi0.c</code> for the <code>spi0_set_clock()</code> helper |
| Slave Selects | 2 — two devices can share the bus                                                                                      |
| Pins          | DIP 35 SCLK (V3) · 36 MOSI (W5) · 37 MISO (V4) · 38 SS0 (U4) · 39 SS1 (V5)                                             |
| Interrupt     | INTC In7                                                                                                               |

**Description:** An SPI master controls external devices (displays, ADCs, flash modules). The interface uses full-duplex push-pull signaling and requires no pull-up resistors; the active device is selected by driving its SS line low. Use the Vitis `xspi` driver. The serial clock is set at runtime through the clock-control block: select a lower rate for breadboard wiring or long cables, and a higher rate for short-wired display modules. A single call to `spi0_set_clock(hz)` takes effect immediately.

**Note:** This is a second, independent SPI controller. Do not confuse it with `axi_quad_spi_0` (0x4050\_0000), which is dedicated to the on-board QSPI boot flash. Firmware should select controllers by instance macro (`XPAR_SPI_0_BASEADDR` vs `XPAR_AXI_QUAD_SPI_0_BASEADDR`), never by generic `XPAR_XSPI_n_*` numbering.

## 5.6 Analog-to-Digital Converter (XADC)

| Item         | Value       |
|--------------|-------------|
| Base Address | 0x4060_0000 |

| Item            | Value                                       |
|-----------------|---------------------------------------------|
| Resolution      | 12-bit                                      |
| Conversion Rate | 500 KSPS aggregate                          |
| Sequencer       | Continuous scan over the 5 enabled channels |

**Enabled Channels:**

| Channel     | Source             | Description                                                 |
|-------------|--------------------|-------------------------------------------------------------|
| VAUX4       | DIP Pin 15 (G3/G2) | External analog input 0 (on-board divider: 0–3.3 V → 0–1 V) |
| VAUX12      | DIP Pin 16 (H2/J2) | External analog input 1 (on-board divider: 0–3.3 V → 0–1 V) |
| Temperature | Internal           | FPGA die temperature monitor                                |
| VCCINT      | Internal           | Core voltage monitor (1.0 V)                                |
| VCCAUX      | Internal           | Auxiliary voltage monitor (1.8 V)                           |

**Description:** The Artix-7 built-in 12-bit ADC measures external analog signals and monitors internal FPGA temperature and supply voltages. External input pins pass through an on-board resistive voltage divider (2.32 kΩ / 1 kΩ, ratio ≈ 0.301), accepting up to 3.3 V at the DIP pin.

**Effective Per-Channel Sampling Rate:** The sequencer continuously cycles through all 5 enabled channels (VAUX4, VAUX12, Temperature, VCCINT, VCCAUX). The aggregate conversion rate is 500 KSPS, giving each channel an effective rate of  $500 \text{ K} \div 5 = 100 \text{ KSPS}$ .

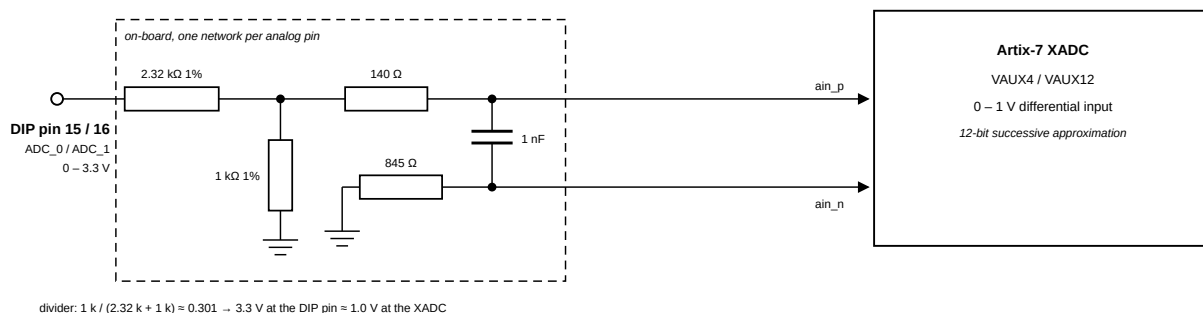
**5.6.1 Analog Input Circuit**

Figure 6: On-board voltage divider circuit for XADC analog inputs

The XADC expects an input range of 0–1 V. The board includes a resistive voltage divider (2.32 kΩ / 1 kΩ, all 1% precision) that scales the DIP pin voltage down to the FPGA's acceptable range. A 140 Ω series resistor and 1 nF capacitor are placed on the differential pair for filtering. The ADx\_N path has an 845 Ω series resistor.

| Parameter                          | Value                                                                             |
|------------------------------------|-----------------------------------------------------------------------------------|
| Max input voltage on DIP pin 15/16 | 3.3 V (relative to GND on pin 25)                                                 |
| Voltage at FPGA after divider      | 0–1 V                                                                             |
| Divider ratio                      | $1 \text{ k} \Omega / (2.32 \text{ k} \Omega + 1 \text{ k} \Omega) \approx 0.301$ |
| Resistor tolerance                 | 1%                                                                                |

**Note:** Pins 15 and 16 are routed through on-board voltage dividers and are dedicated analog inputs; they are not available as digital I/O. Do not exceed 3.3 V on these pins.

## 6 Electrical Characteristics and Power

### 6.1 Electrical Characteristics

| Parameter                       | Value                                                                                                    |
|---------------------------------|----------------------------------------------------------------------------------------------------------|
| I/O Standard                    | LVC MOS33 (3.3 V)                                                                                        |
| Series Resistance on DIP Pins   | None                                                                                                     |
| Max Input Voltage (digital I/O) | abs. max $-0.4\text{ V}$ to $V_{CCO} + 0.55\text{ V} = \mathbf{3.85\text{ V}}$ (DS181); not 5 V tolerant |
| Analog Input Range (Pin 15, 16) | 0–3.3 V (on-board divider scales to the XADC's 0–1 V; see Section 5.6.1)                                 |
| Clock Source                    | 12 MHz oscillator (FPGA pin L17), PLL → 100 MHz                                                          |

- When a USB host is attached, the VU pin is driven to USB voltage (4.5–5.5 V). Any external power source on VU **must be disconnected** before USB is connected, or the external supply may be damaged.
- This is **especially dangerous with battery power sources**, as batteries cannot tolerate reverse current.
- The FPGA I/O pins on the DIP connector have no series resistors. The Artix-7 absolute maximum input rating is  $V_{CCO} + 0.55\text{ V} = \mathbf{3.85\text{ V}}$  (and  $-0.4\text{ V}$  minimum, per DS181). The pins are **not 5 V tolerant**; use a level shifter or divider for 5 V devices.

### 6.2 Power Architecture

The Cmod A7-35T uses a Linear Technology LTC3569 triple-output buck regulator (IC10) to generate all onboard supply voltages from a single input rail called VU.

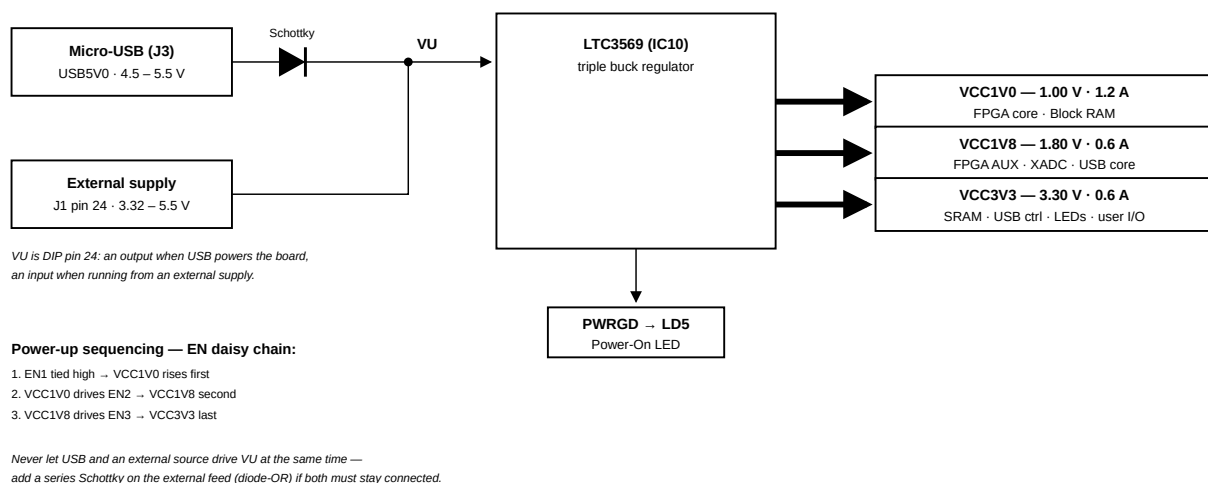


Figure 7: Cmod A7 power supply tree

### 6.3 Power Input Options

The board can be powered from either of the following sources:

| Source          | Connector       | Schematic Net | Min Voltage | Recommended | Max Voltage |
|-----------------|-----------------|---------------|-------------|-------------|-------------|
| USB             | J3 (Micro-USB)  | USB5V0        | 4.5 V       | 5.0 V       | 5.5 V       |
| External Supply | J1 Pin 24 (DIP) | VU            | 3.32 V*     | 5.0 V       | 5.5 V       |

**Note:** \* The minimum external voltage on VU depends on the current drawn from the VCC3V3 rail via the Pmod header:

- 0 mA drawn: minimum 3.32 V
- 100 mA drawn: minimum 3.38 V
- 250 mA drawn: minimum 3.48 V

The GND reference is on J1 Pin 25.

### 6.4 Output Power Rails

All three rails are generated by the LTC3569 from the VU input.

| Rail   | Typical Voltage | Min / Max Voltage | Max Current | Powered Devices                                          |
|--------|-----------------|-------------------|-------------|----------------------------------------------------------|
| VCC1V0 | 1.00 V          | 0.95 V / 1.05 V   | 1.2 A       | FPGA Core, Block RAM                                     |
| VCC1V8 | 1.80 V          | 1.71 V / 1.89 V   | 0.6 A       | FPGA AUX, ADC, USB Core                                  |
| VCC3V3 | 3.30 V          | 3.135 V / 3.465 V | 0.6 A       | SRAM, USB Controller, Pmod, LEDs, Buttons, FPGA User I/O |

### 6.4.1 Power-Up Sequence

The LTC3569 enable pins are daisy-chained to enforce a sequential startup:

1. VCC1V0 starts first (EN1 is tied high).
2. VCC1V8 starts after VCC1V0 is stable (EN2 is driven by VCC1V0).
3. VCC3V3 starts after VCC1V8 is stable (EN3 is driven by VCC1V8).

The PWRGD (Power Good) output drives the Power On LED (LD5).

## 6.5 VU Pin Behavior

| Condition         | VU Pin Function                                                                              |
|-------------------|----------------------------------------------------------------------------------------------|
| USB connected     | Output. Driven from USB 5 V through a Schottky diode; approximately 5 V after the diode drop |
| USB not connected | Input. Accepts an external supply of 3.32–5.5 V                                              |

## 6.6 Dual Power Source (USB + External)

The on-board Schottky diode is located on the USB side (between USB VBUS and VU), not on the VU input pin. The resulting behavior is as follows:

| Scenario                                                            | Safe? | Notes                                                |
|---------------------------------------------------------------------|-------|------------------------------------------------------|
| USB only                                                            | ✓     | Normal operation                                     |
| External only (VU pin)                                              | ✓     | Normal operation                                     |
| Both connected, only one active                                     | ✓     | The inactive source supplies no current, no conflict |
| Both connected <b>and both supplying power</b>                      | ✗     | Two voltage sources fight on the same VU node        |
| Both connected and both active, <b>with external Schottky added</b> | ✓     | Higher-voltage source wins; lower is blocked         |

**Note:** Physically connecting both cables is not itself hazardous. The hazard arises when both sources actively supply voltage at the same time without isolation.

If both sources must be connected and powered simultaneously (e.g., USB for JTAG/UART while operating from an external supply), add a Schottky diode in series with the external supply on the VU pin. This creates a diode-OR circuit in which the source with the higher voltage supplies the load and the other source is blocked.

## 6.7 Quick Reference for External Power Design

| Parameter                           | Value                            |
|-------------------------------------|----------------------------------|
| Input Connector                     | J1 Pin 24 (VU) + Pin 25 (GND)    |
| Recommended Input Voltage           | 5.0 V                            |
| Acceptable Input Range              | 3.32 – 5.5 V                     |
| Total Max Current (all rails)       | ~2.4 A (1.2 + 0.6 + 0.6)         |
| Power Good Indicator                | LD5 (LED)                        |
| Regulator IC                        | LTC3569                          |
| Protection Required for Dual Supply | Schottky diode in series with VU |

## 7 JTAG Debug Mode

This chapter describes how to load and debug a MicroBlaze RISC-V application over JTAG using the AMD Vitis Unified IDE. JTAG loads the program directly into RAM and provides full debug control. Nothing is written to flash.

### 7.1 Prerequisites

- The hardware design as an .xsa file: use the prebuilt `release/top_wrapper.xsa`
- Vitis Unified IDE 2025.2 installed
- Cmod A7-35T board connected via its micro-USB cable (the cable carries power, UART, and JTAG)
- A serial terminal program (GTKTerm, PuTTY, or pyserial's miniterm) for the board's console output

### 7.2 Open the Workspace

Launch the Vitis Unified IDE and click **Set Workspace** on the Welcome page.

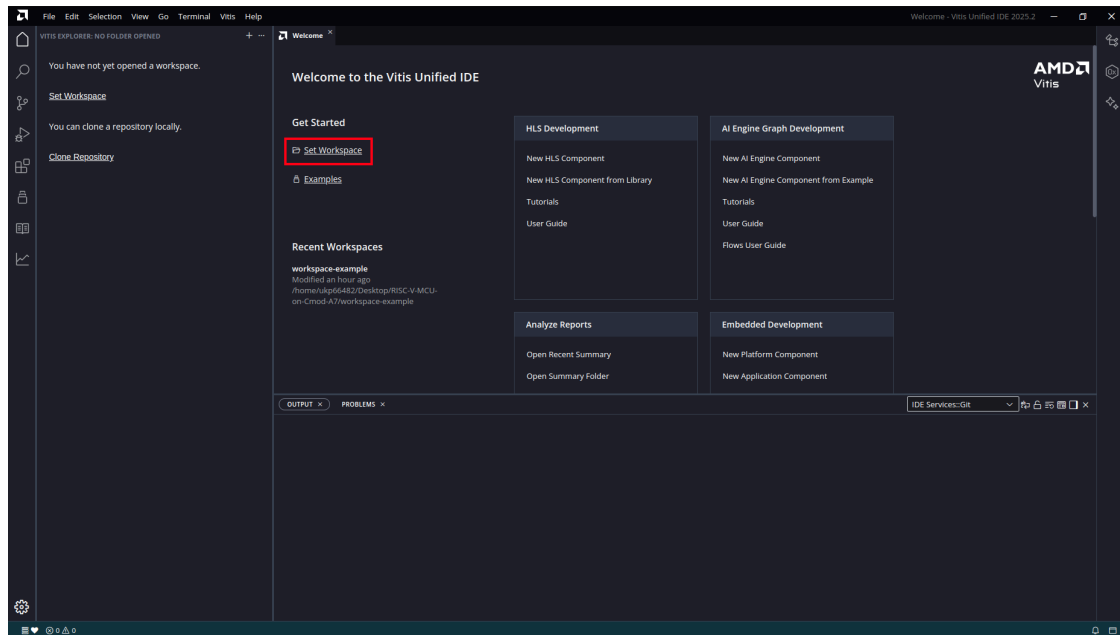


Figure 8: Set Workspace

Select the `workspace-example/` folder in the repository. The template paths in this guide assume this location; any empty folder can also be used if the paths are adjusted accordingly.

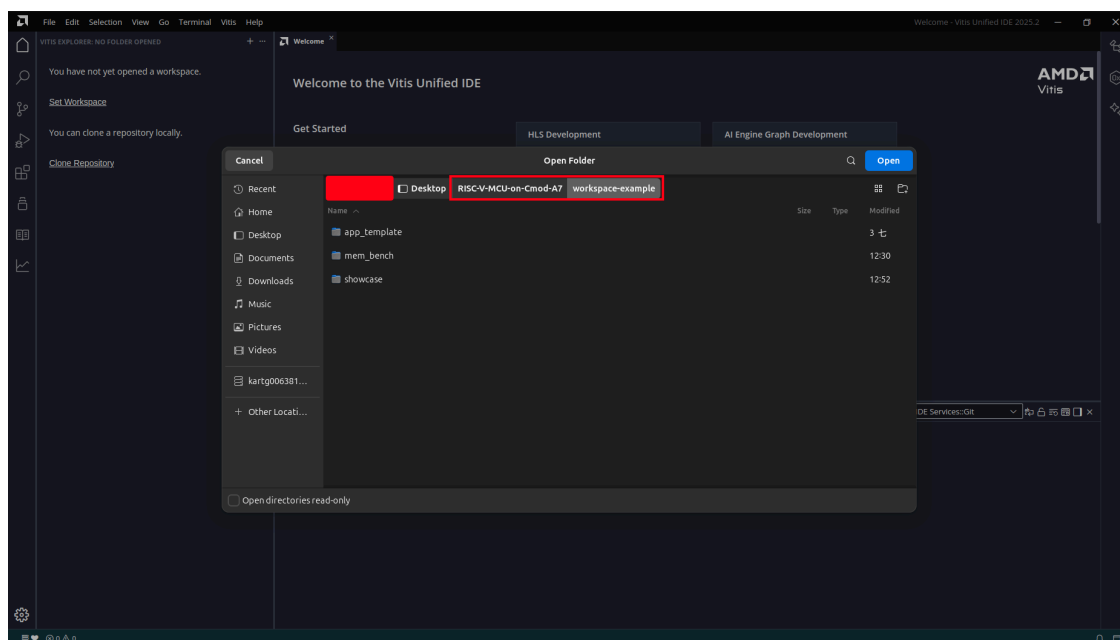


Figure 9: Open Folder

The first time a Vitis installation opens this workspace it may show an **Update Workspace** dialog. Click **Update**. This action refreshes only the workspace metadata and does not modify any component.

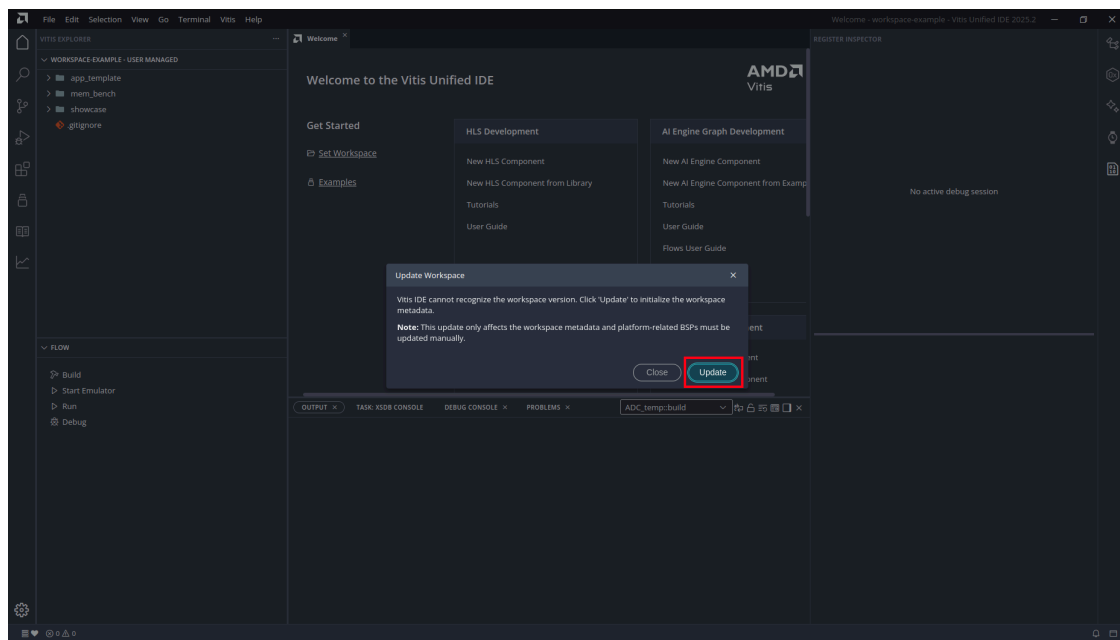


Figure 10: Update Workspace

## 7.3 Create a Platform

The platform wraps the hardware design and provides the driver layer (BSP). It is created once per workspace and reused by every application. If the workspace already contains a platform component, skip to Section 7.4.

### 7.3.1 New Platform Component

From the menu bar, select **File > New Component > Platform**.

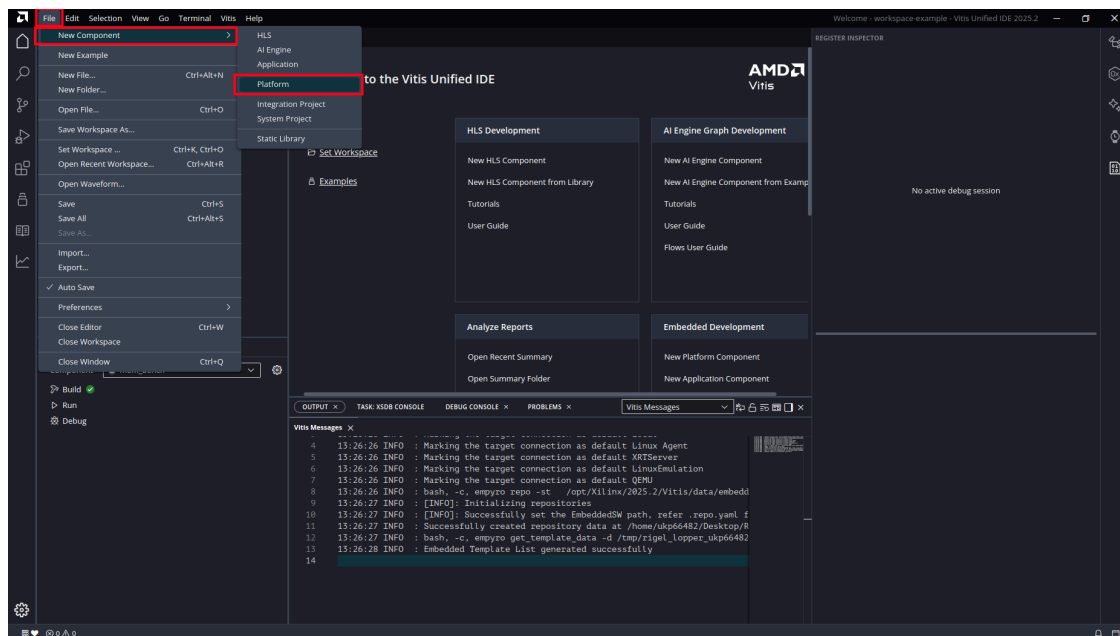


Figure 11: File > New Component > Platform

### 7.3.2 Platform Name and Location

Enter `platform` as the component name and keep the workspace directory as the location. Click **Next**.

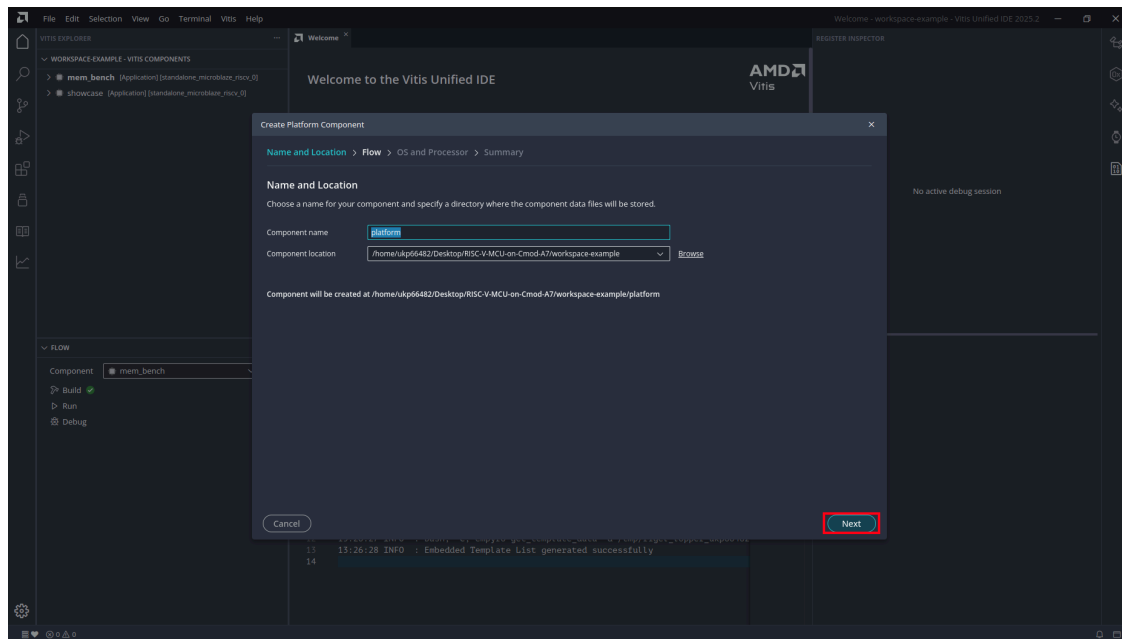


Figure 12: Platform Name and Location

### 7.3.3 Select the Hardware Design (XSA)

On the **Flow** page, select **Hardware Design** and click **Browse** next to the *Hardware Design (XSA) For Implementation* field.

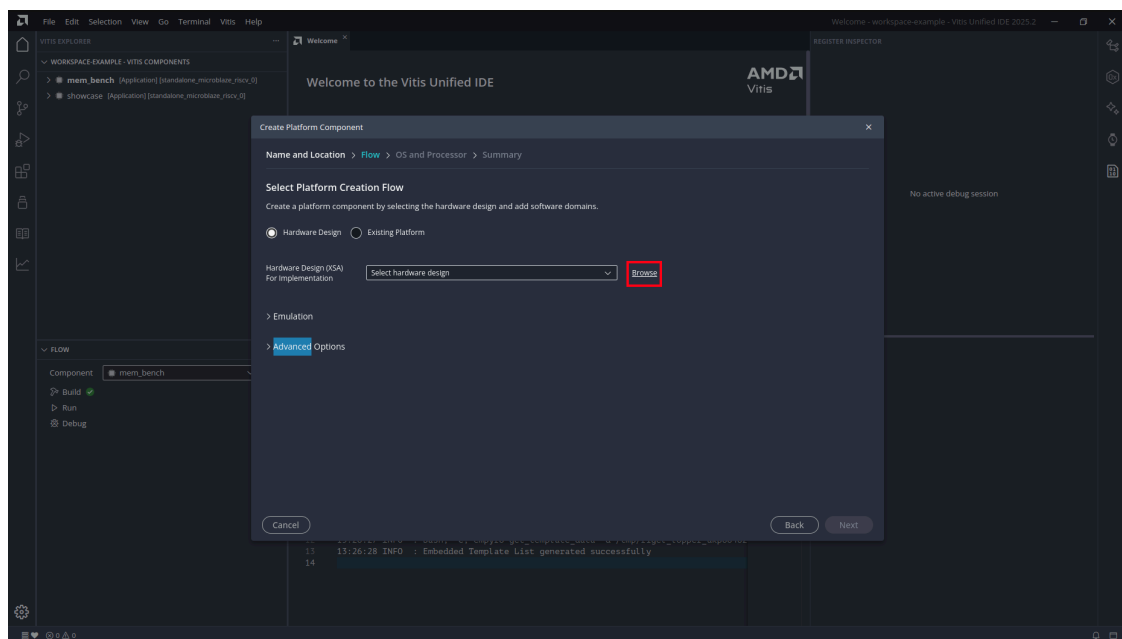


Figure 13: Select Platform Creation Flow

Select the prebuilt `release/top_wrapper.xsa` from the repository.

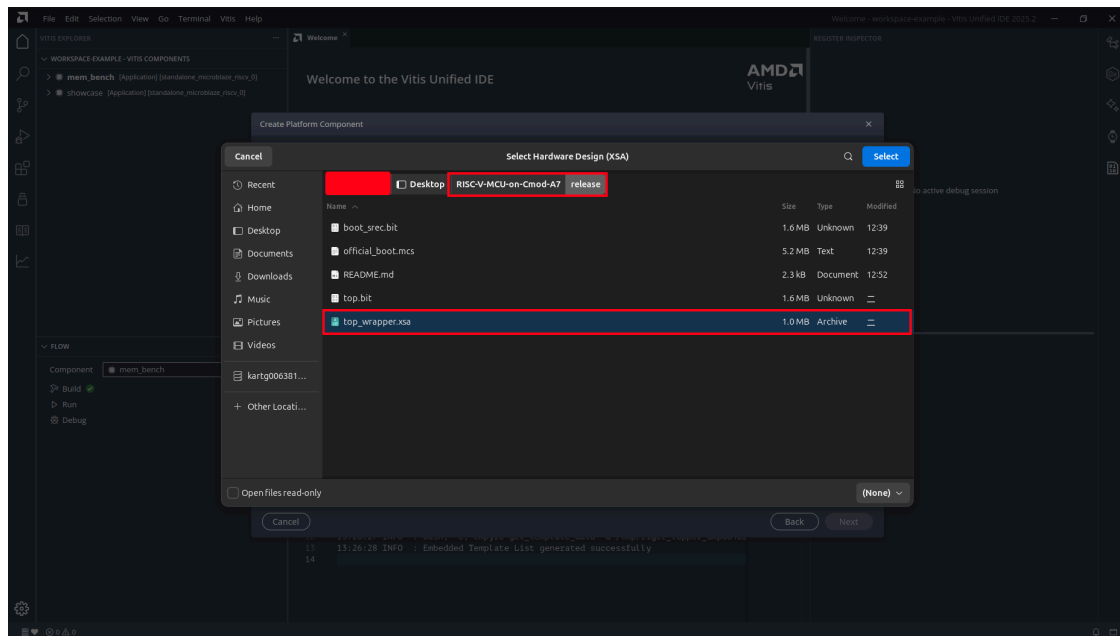


Figure 14: Select top\_wrapper.xsa

### 7.3.4 Select Operating System and Processor

- **Operating system:** standalone
- **Processor:** microblaze\_riscv\_0

Click **Next**.

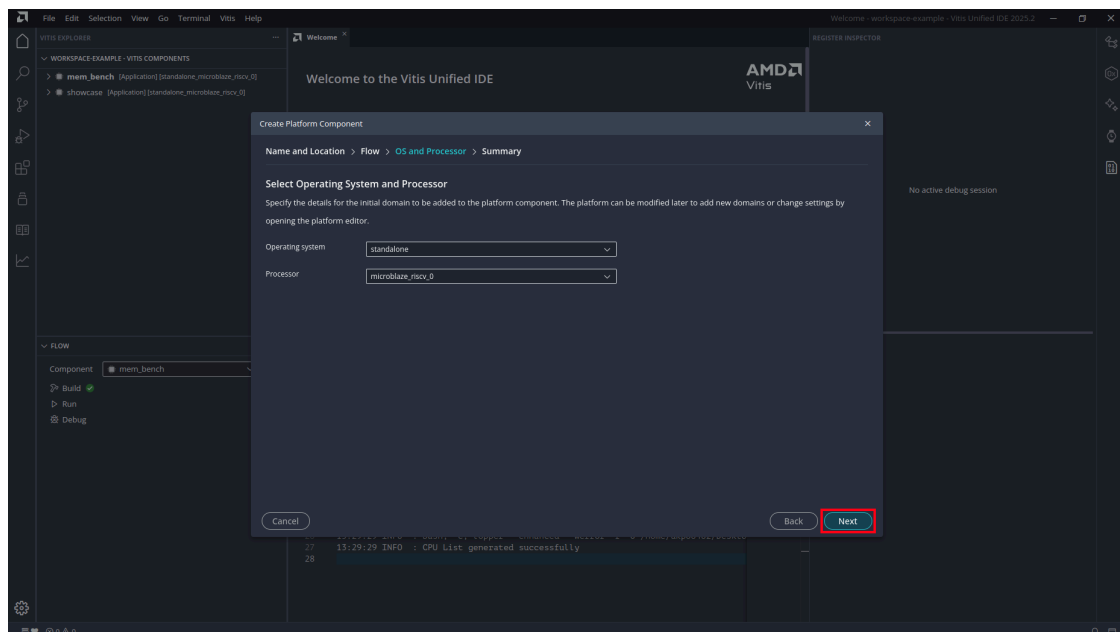


Figure 15: OS and Processor

### 7.3.5 Platform Summary

Review the settings and click **Finish**.

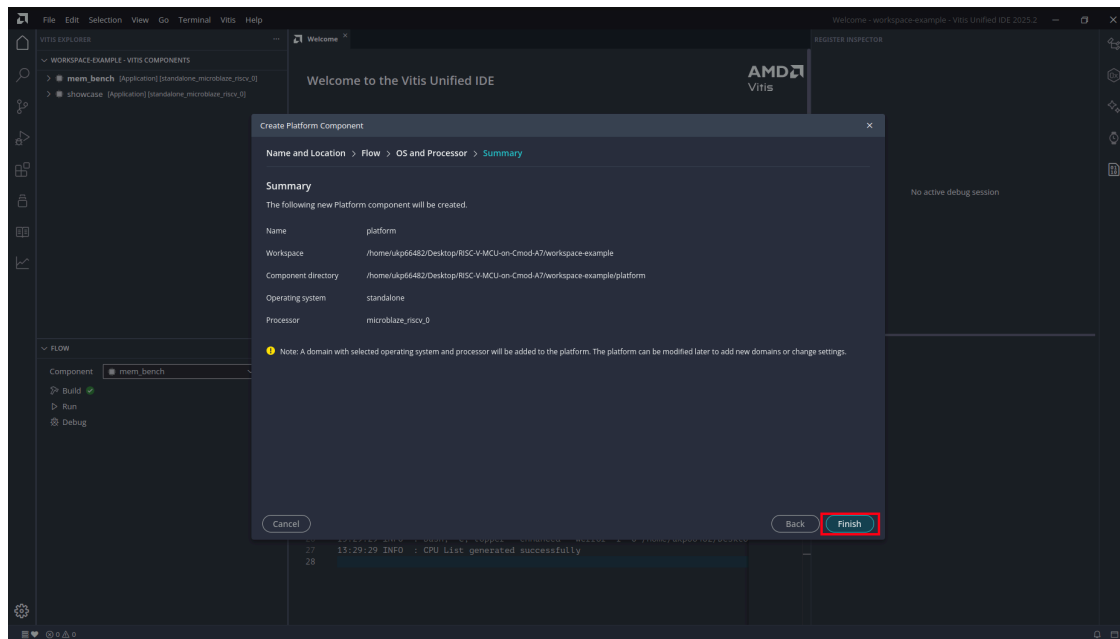


Figure 16: Platform Summary

Vitis generates the platform component. When it finishes, the Explorer shows the platform component with its Sources and Output trees, and the notification “Created platform: platform” appears.

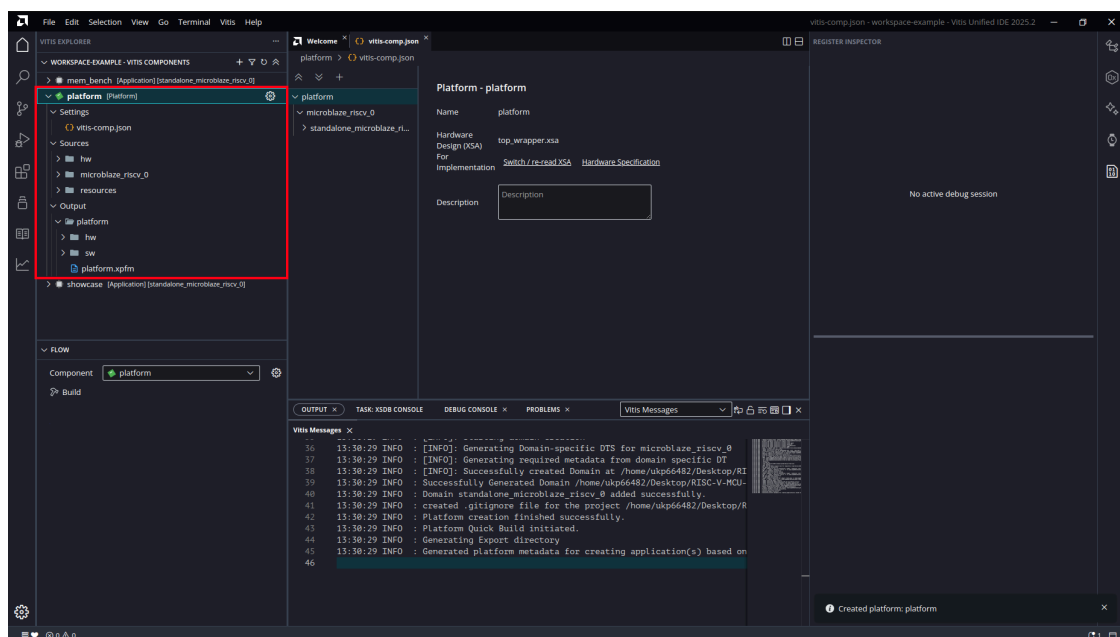


Figure 17: Platform Created

## 7.4 Create an Application

### 7.4.1 Create from Template

Open the **Examples** view from the left activity bar. Under *Embedded Software Examples*, select **Hello World** and click the + (Create Application Component from Template) button.

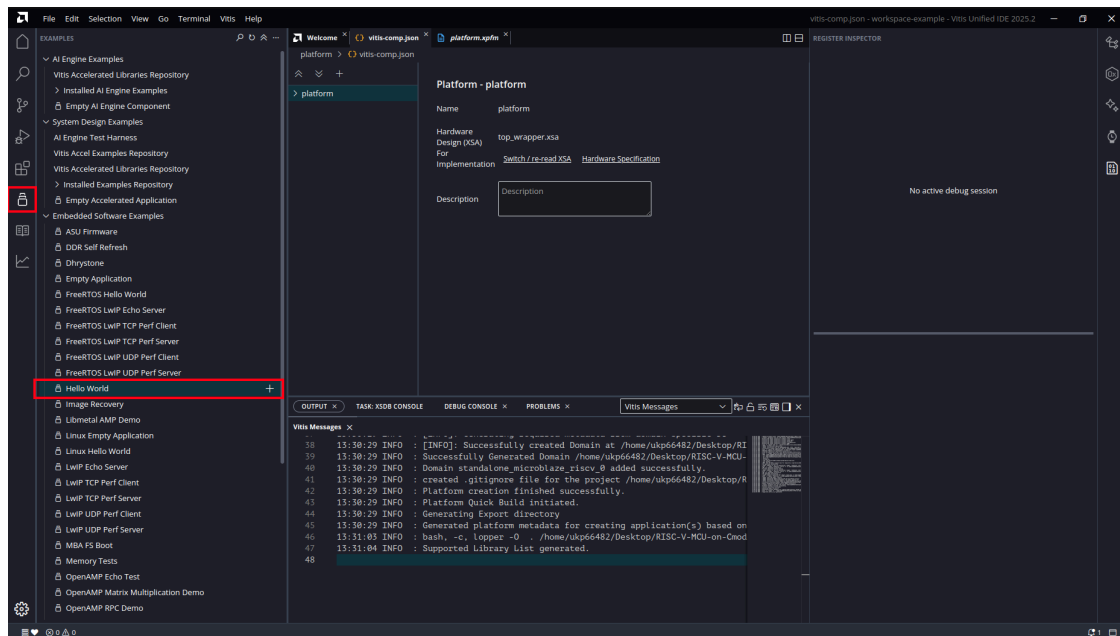


Figure 18: Hello World Template

## 7.4.2 Application Name and Location

Enter your application name (this guide uses test). Click **Next**.

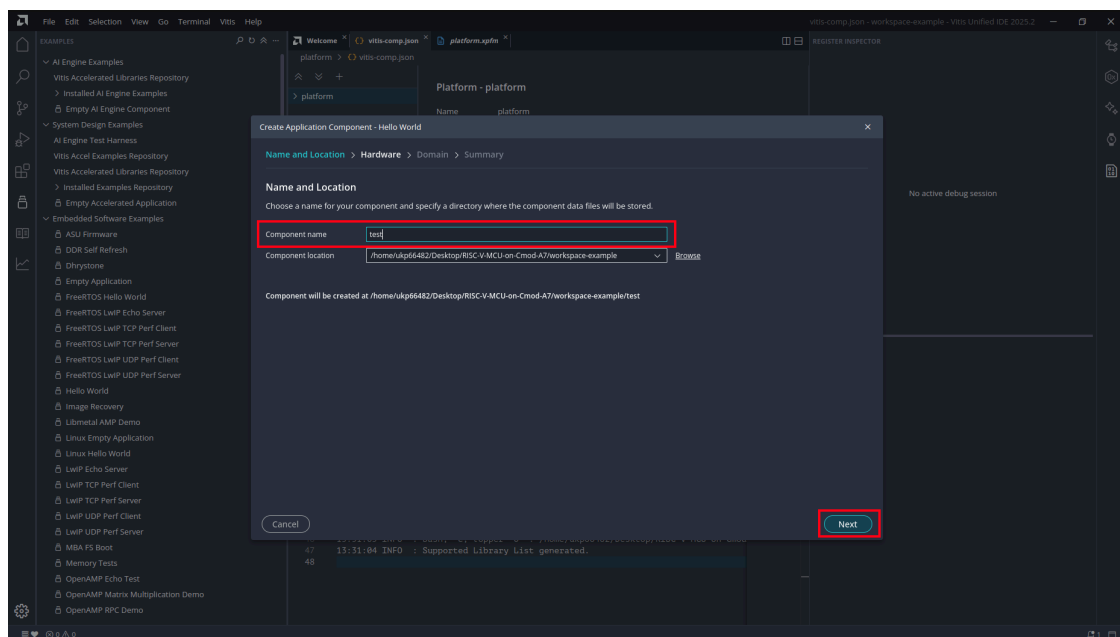


Figure 19: Application Name

## 7.4.3 Select the Platform

Select the platform component created in Section 7.3 (Board: cmod\_a7-35t). Click **Next**.

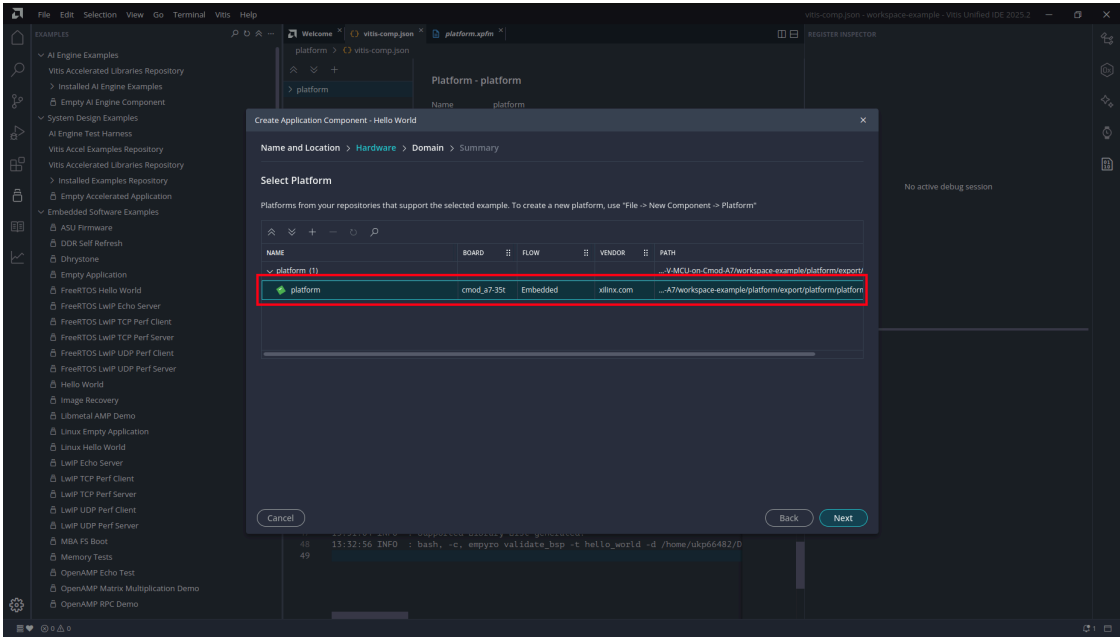


Figure 20: Select Platform

7.4.4 Select the Domain

Select standalone\_microblaze\_riscv\_0. Click Next.

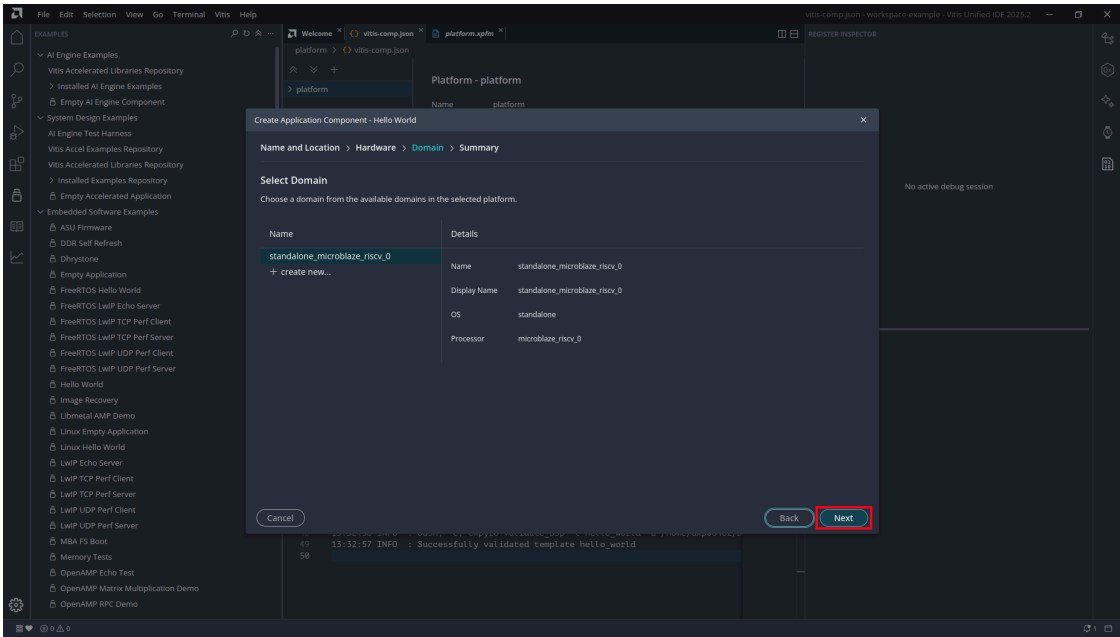


Figure 21: Select Domain

7.4.5 Application Summary

Review the settings and click Finish.

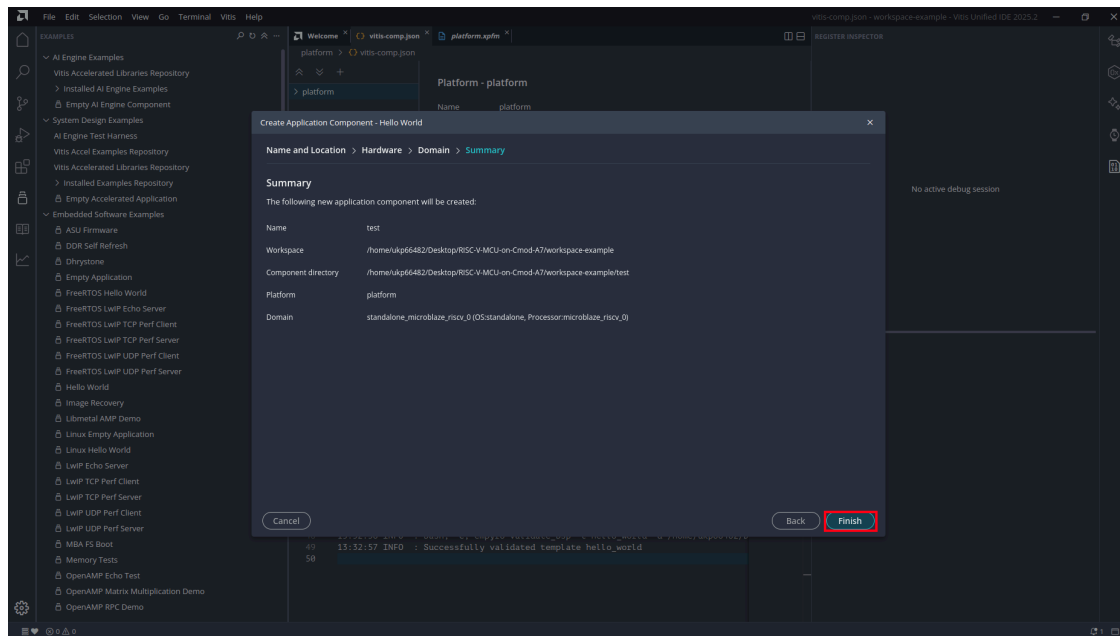


Figure 22: Application Summary

The new component appears in the Explorer. Its `src/` folder holds the template's generated files: `helloworld.c`, `lscript.ld`, `platform.c`, and `platform.h`.

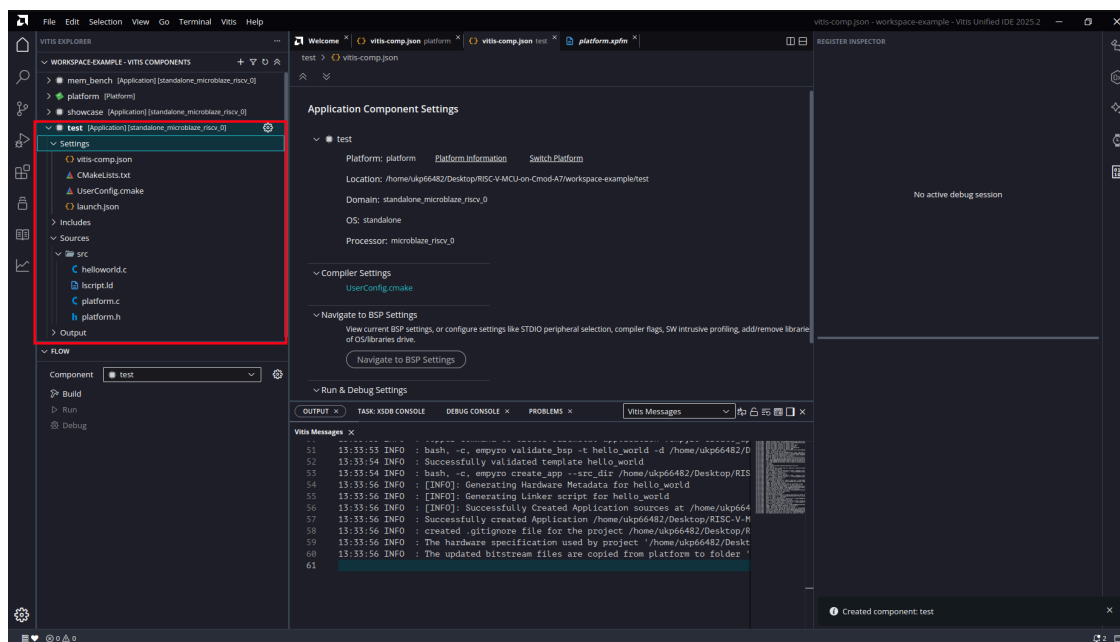


Figure 23: Application Created

## 7.5 Replace the Sources with the Course Template

The generated Hello World sources are replaced entirely by the course template. The template `main.c` starts with `mcu_init()`, and its `lscript.ld` places code in the 512 KB SRAM, places the stack in DTCM, and provides the `ITCM_FUNC/DTCM_DATA` fast-memory attributes.

### 7.5.1 Delete the Generated Files

In the Explorer, select all four files under `src/` (`helloworld.c`, `lscript.ld`, `platform.c`, `platform.h`), right-click, and choose **Delete**.

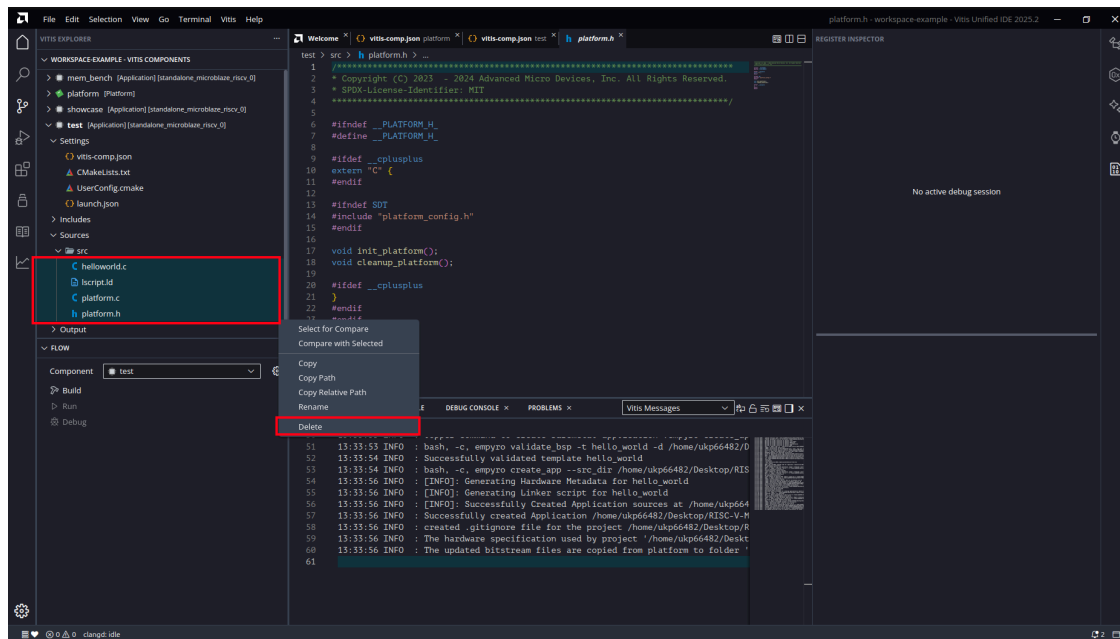


Figure 24: Delete Generated Sources

## 7.5.2 Import the Template Files

Right-click the `src/` folder and choose **Import > Files...**

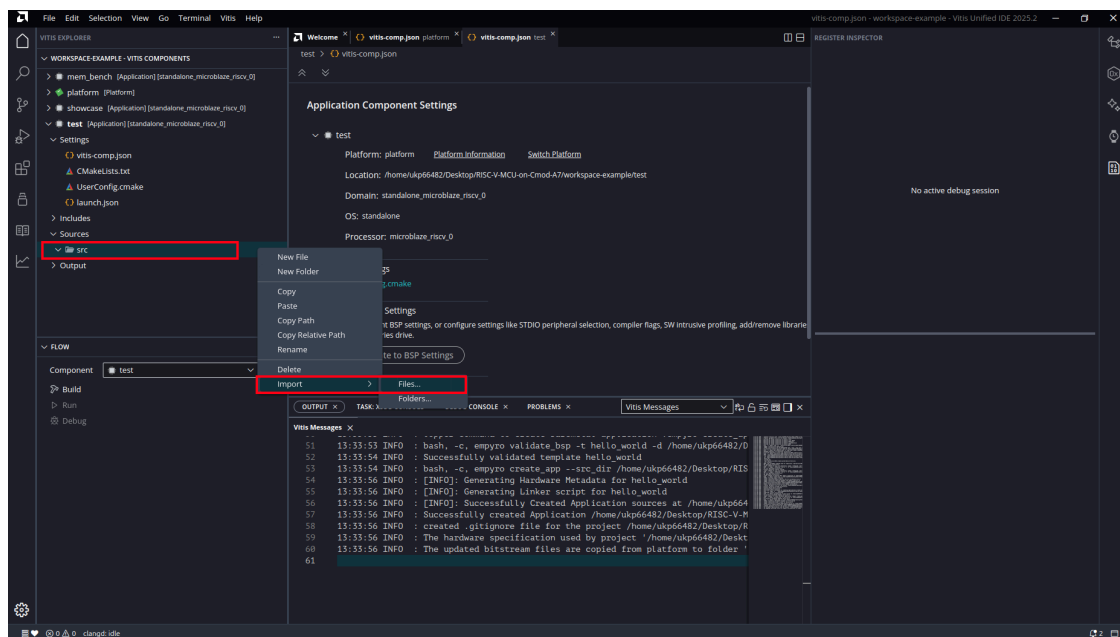


Figure 25: Import Files

Navigate to `workspace-example/app_template/src/` and select both `main.c` and `lscript.ld`. Click **Open**.

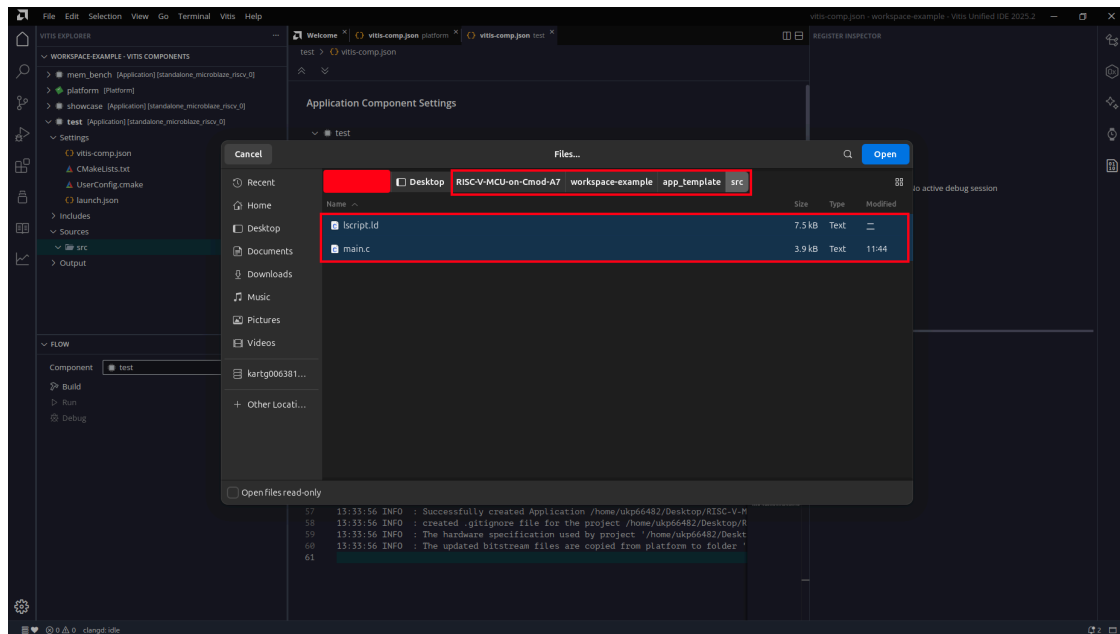


Figure 26: Select Template Files

Afterwards `src/` contains exactly `main.c` and `lscript.ld`. Files inside `src/` are compiled automatically; no build configuration is needed.

The `mcu_init()` call must remain the first line of `main()`. It copies ITCM\_FUNC code into the ITCM, zeroes the DTCM, and sets the USB UART to 115200. The demo loop then cycles the on-board RGB LED and prints a status line about once a second.

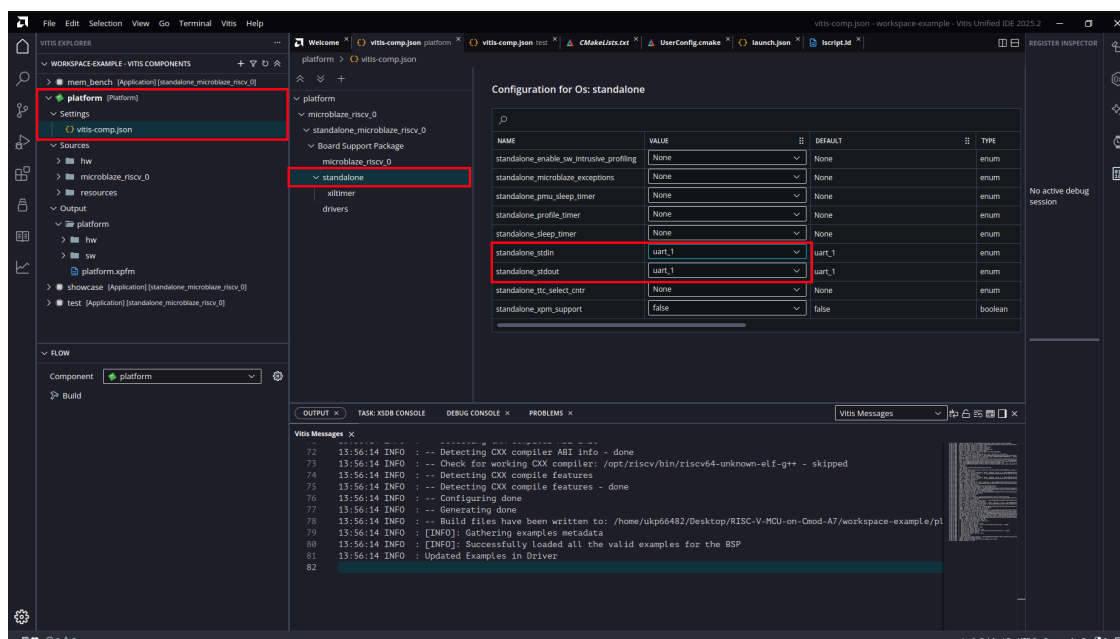
### 7.5.3 About the Linker Script

The imported `lscript.ld` defines three memory regions: `sram` (512 KB), `itcm` (32 KB), and `dtcm` (64 KB). It maps everything to SRAM except the stack (DTCM) and ITCM\_FUNC code (ITCM). The component settings page *Linker Script* can display the regions, but treat it as read-only: it understands only part of the hand-written script (an incomplete section list is expected), and letting it regenerate the script would discard the ITCM/DTCM layout. To resize the stack or heap, edit `_STACK_SIZE` / `_HEAP_SIZE` at the top of `lscript.ld` instead.

## 7.6 Set the Console UART (BSP)

The BSP determines which UART carries `xil_printf` output. The default is `uart_1` (the DIP-header pins); with the default setting, no output appears on the USB console.

Open the platform's **Settings** > **vitis-comp.json**, navigate to **standalone** (Board Support Package > `microblaze_riscv_0` > `standalone`). The configuration table shows `standalone_stdin` and `standalone_stdout` set to `uart_1`:

Figure 27: BSP Defaults to `uart_1`

Change both to `uart_USB`. The output log reports the domain being reconfigured.

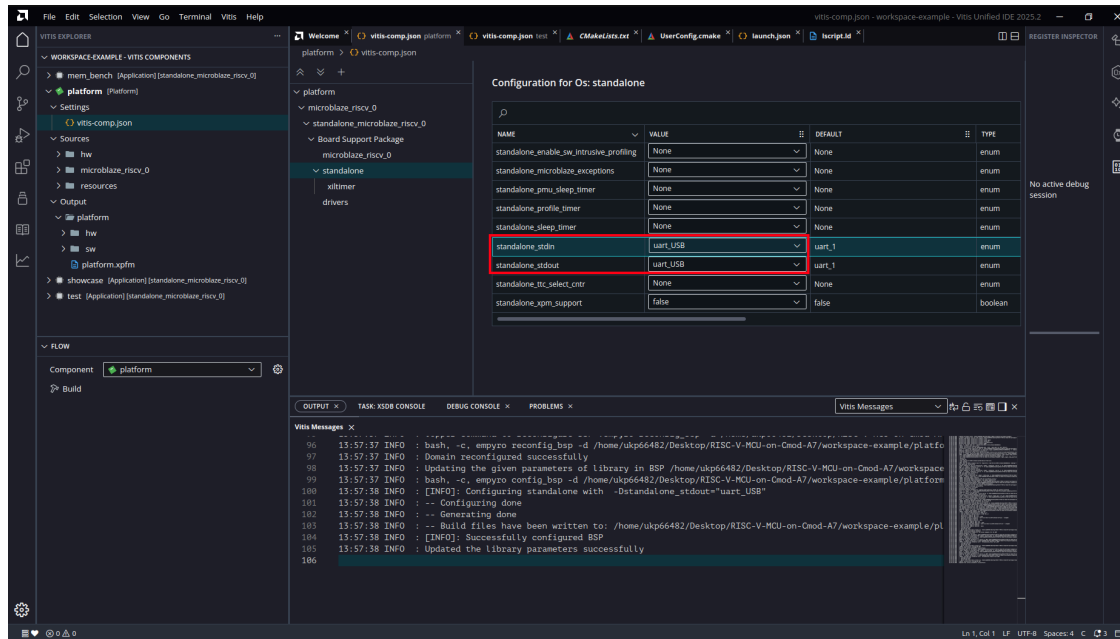


Figure 28: stdin/stdout Set to `uart_USB`

**Note:** If the platform BSP is ever regenerated (for example after changing another BSP option), re-check this page; regeneration can reset stdin/stdout to `uart_1`. After changing it, rebuild both the platform and the application; the setting is compiled into the application's BSP library.

## 7.7 Build

### 7.7.1 Build the Platform

In the **FLOW** panel, select the **platform** component and click **Build**. Wait for “Platform Build Finished successfully”.

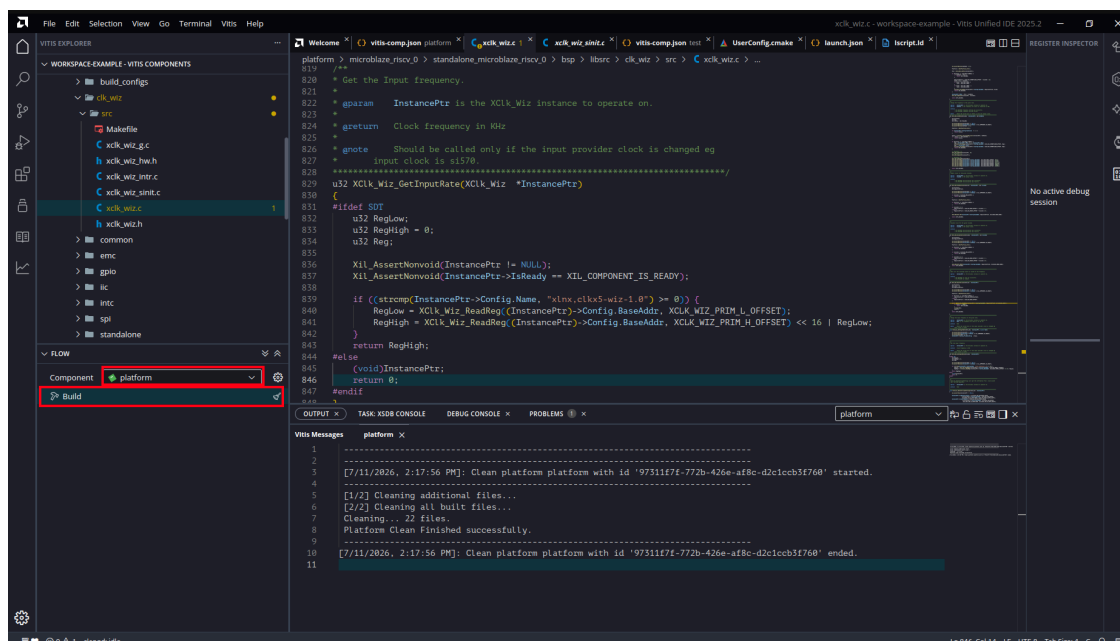


Figure 29: Build Platform

### 7.7.2 Build the Application

Switch the **FLOW** component to your application and click **Build**.

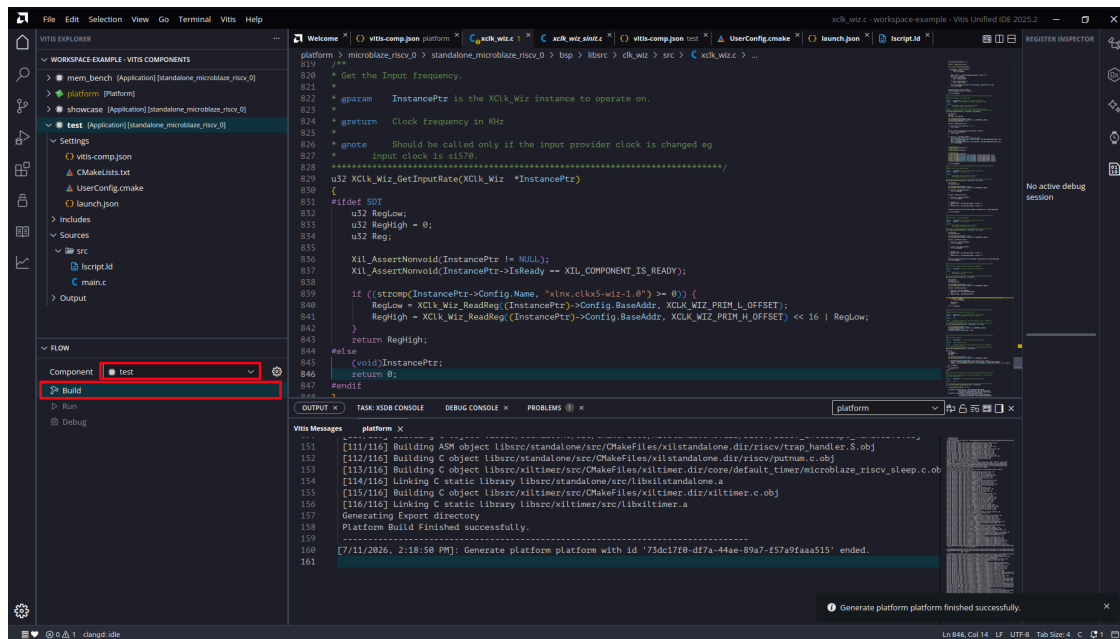


Figure 30: Build Application

The first application build asks how platform builds should be handled. Select **Always build platform with application** and click **Save in Workspace Preference**. BSP changes (such as Section 7.6) then propagate automatically on every build.

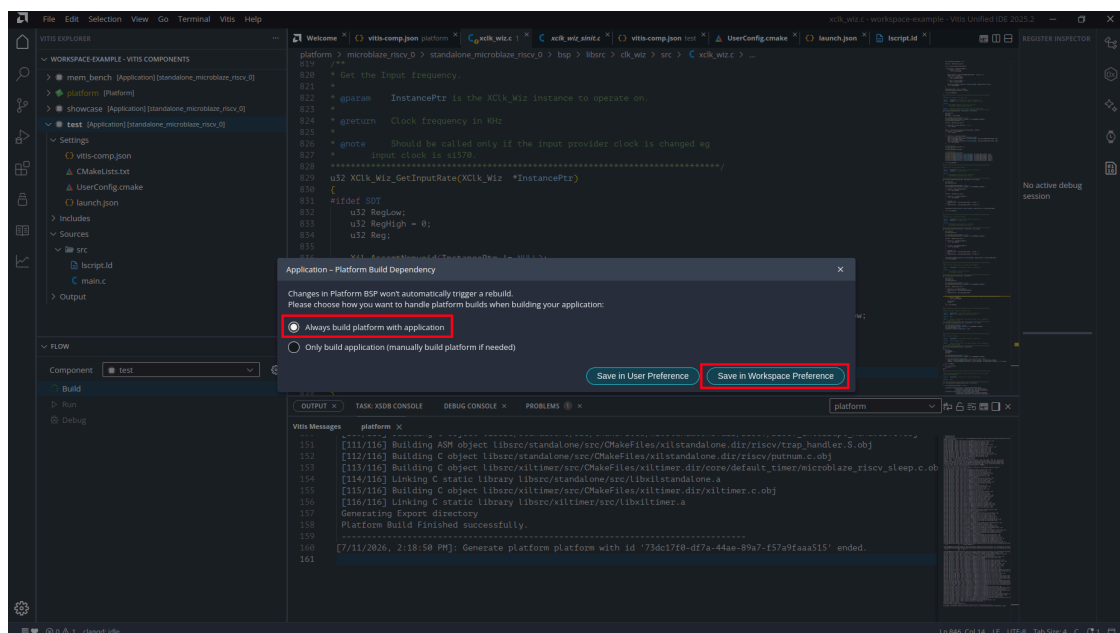


Figure 31: Platform Build Dependency

The build produces `<workspace>/test/build/test.elf`.

**Note:** During the build, Vitis automatically links in the BSP's `boot.S` and the toolchain C library's startup object `crt0.o`: `boot.S` runs first after reset (initializes registers, stack pointer, and trap vector), and `crt0.o` clears `.bss` before calling `main()`.

## 7.8 Debug over JTAG

### 7.8.1 Start a Debug Session

In the **FLOW** panel, click **Debug**. Vitis programs the FPGA over JTAG, loads the ELF into memory, and pauses execution at `main()`.

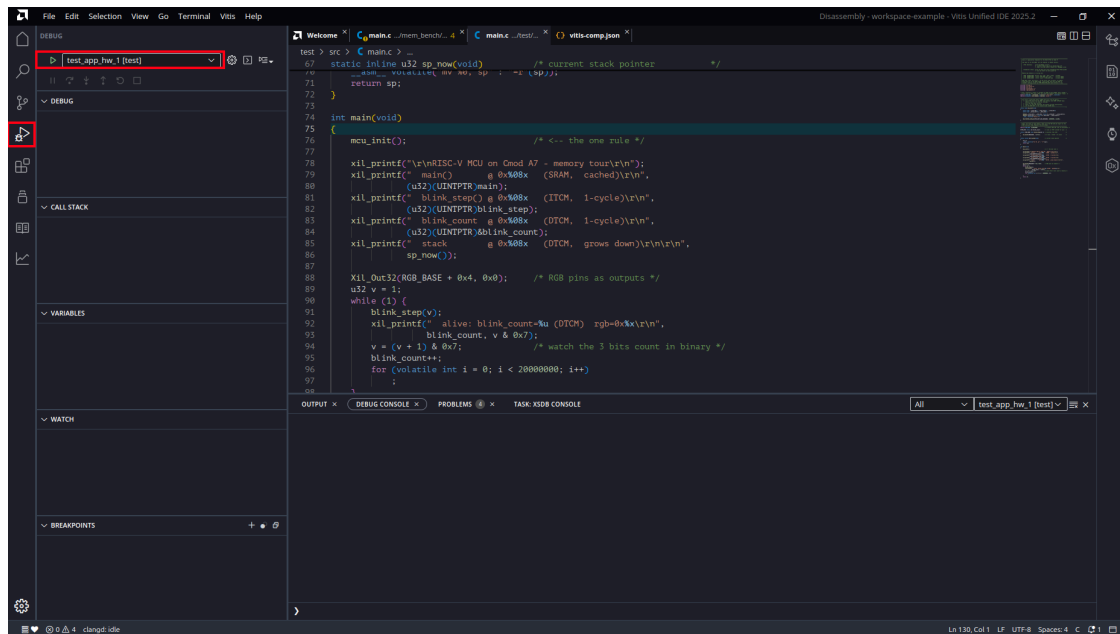
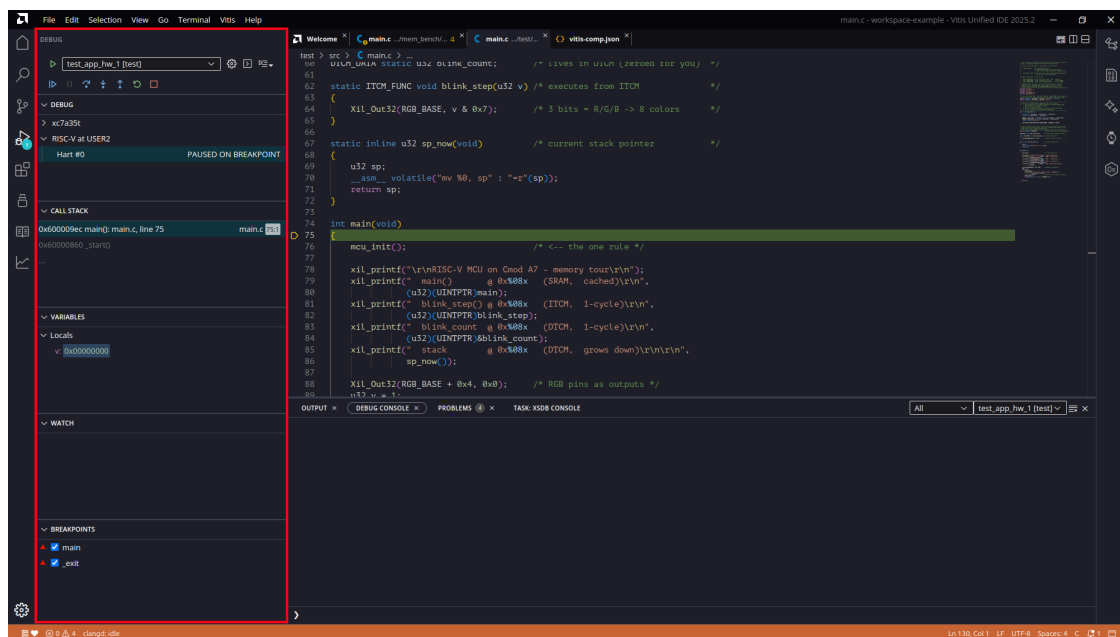


Figure 32: Debug Launched

The Debug view shows the target (*RISC-V at USER2, Hart #0 — PAUSED ON BREAKPOINT*), the call stack, local variables, and breakpoints. Note the call stack address: `main()` is located at `0x600009EC`, in SRAM, exactly where the linker script placed it.

Figure 33: Paused at `main()`

## 7.8.2 Open a Serial Terminal, Then Continue

Open a serial terminal before resuming execution. The board enumerates as two serial ports; the UART is usually the higher-numbered port. Set the terminal to 115200 8N1.

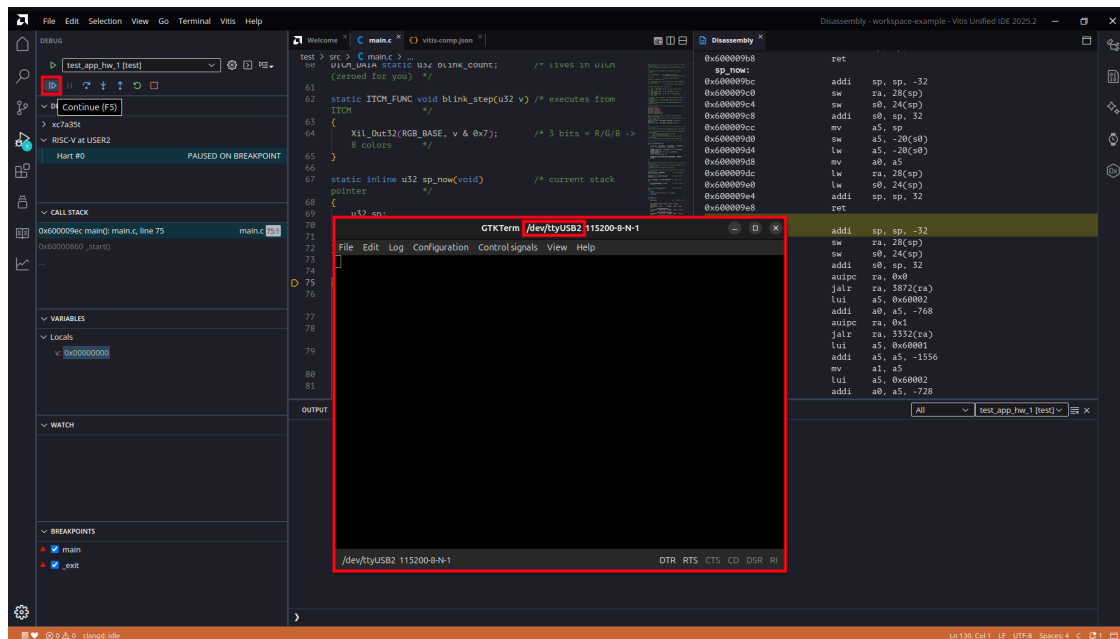


Figure 34: GTKTerm on the Board UART

Click **Continue (F5)**. The application starts: the memory tour prints once, followed by a status line about once a second, and the on-board RGB LED cycles through its colors.

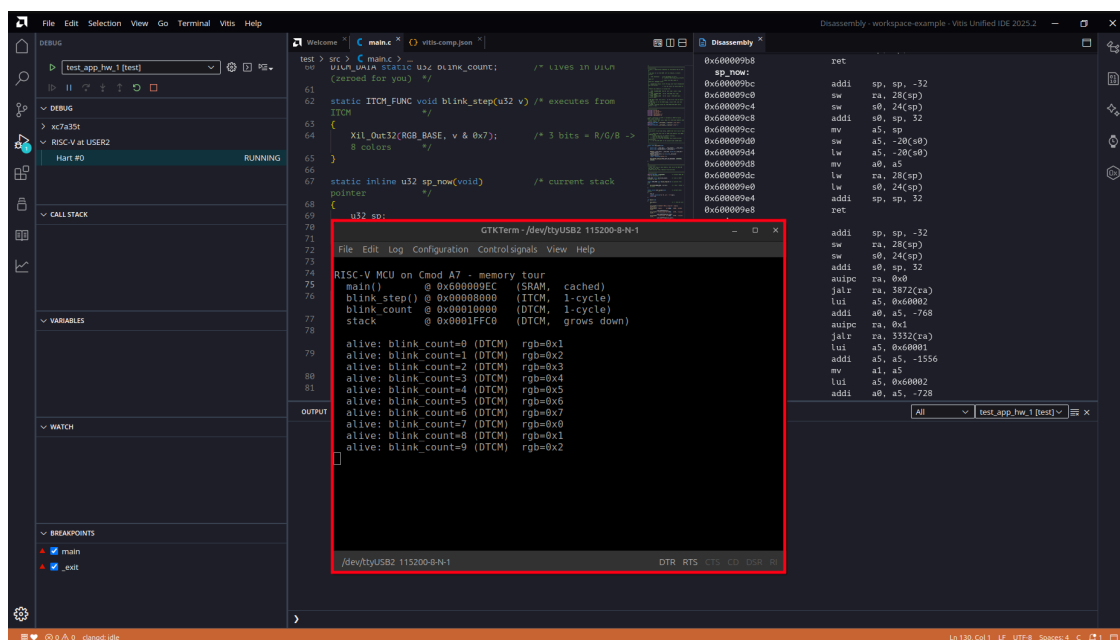


Figure 35: Memory Tour and Status Output

The memory tour output confirms that the memory layout is correct:

```
RISC-V MCU on Cmod A7 - memory tour
main()      @ 0x600009EC (SRAM,  cached)
blink_step() @ 0x00008000 (ITCM,  1-cycle)
blink_count @ 0x00010000 (DTCM,  1-cycle)
stack       @ 0x0001FFC0 (DTCM,  grows down)
```

Each line matches the memory map in the datasheet: code in SRAM, fast code in ITCM, fast data and the stack in DTCM. If no output appears on the terminal, see Section 7.10.

### 7.8.3 Breakpoints and Stepping

Click in the gutter next to a line number to set a breakpoint. The toolbar provides **Continue**, **Step Over**, **Step Into**, and **Step Out**. The left panel shows the call stack, variables, and watch expressions while the target is paused.

To run the application without the debugger, use **Run** in the FLOW panel instead. The loading sequence is the same, but execution does not pause at `main()`.

## 7.9 Inspection Tools

While a debug session is active, the **View** menu provides three inspectors:

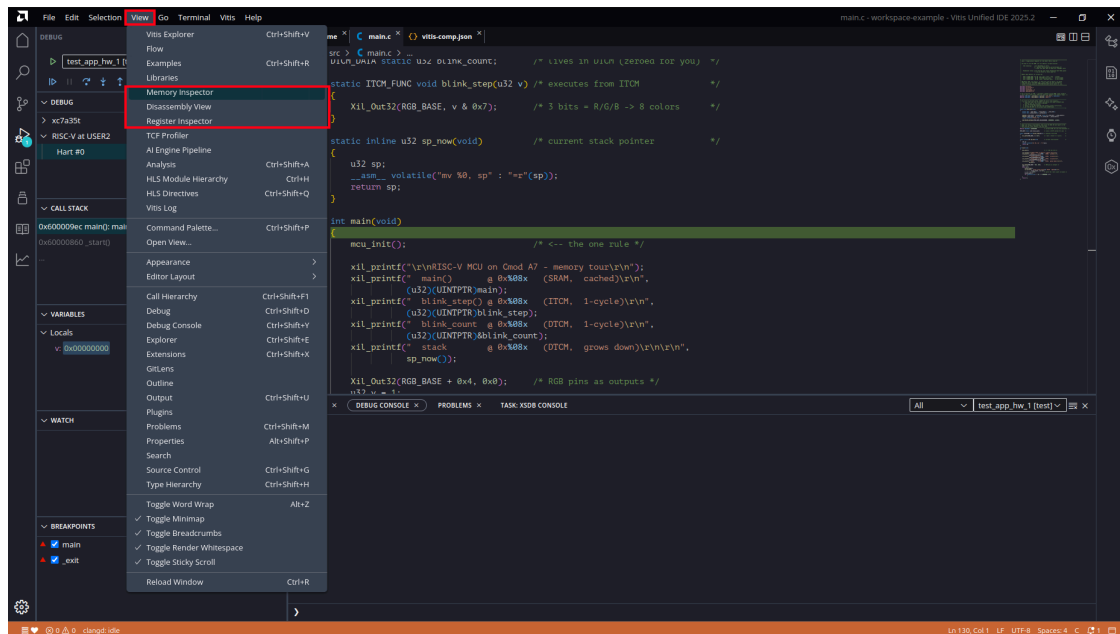


Figure 36: View Menu

### 7.9.1 Memory Inspector

Enter an address to inspect memory directly, for example `0x60000000` (the application image in SRAM), `0x8000` (ITCM), or `0x10000` (DTCM).

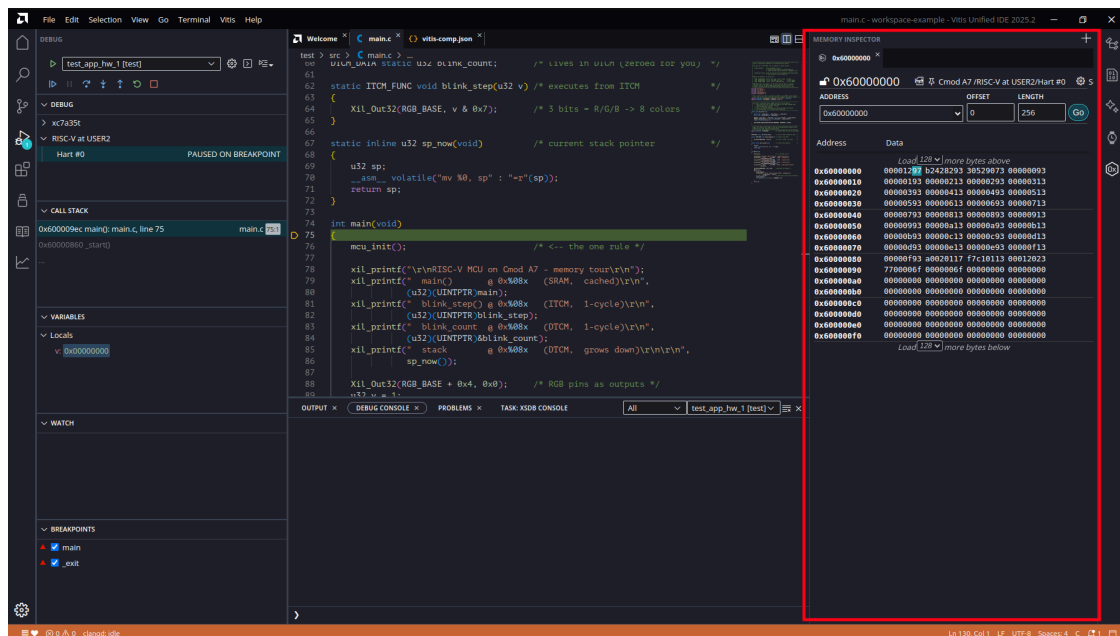


Figure 37: Memory Inspector

### 7.9.2 Register Inspector

The Register Inspector shows all processor registers with live values. With the target paused at `main()`, `sp` reads `0x00020000`, the top of the DTCM, matching the linker script's stack placement.

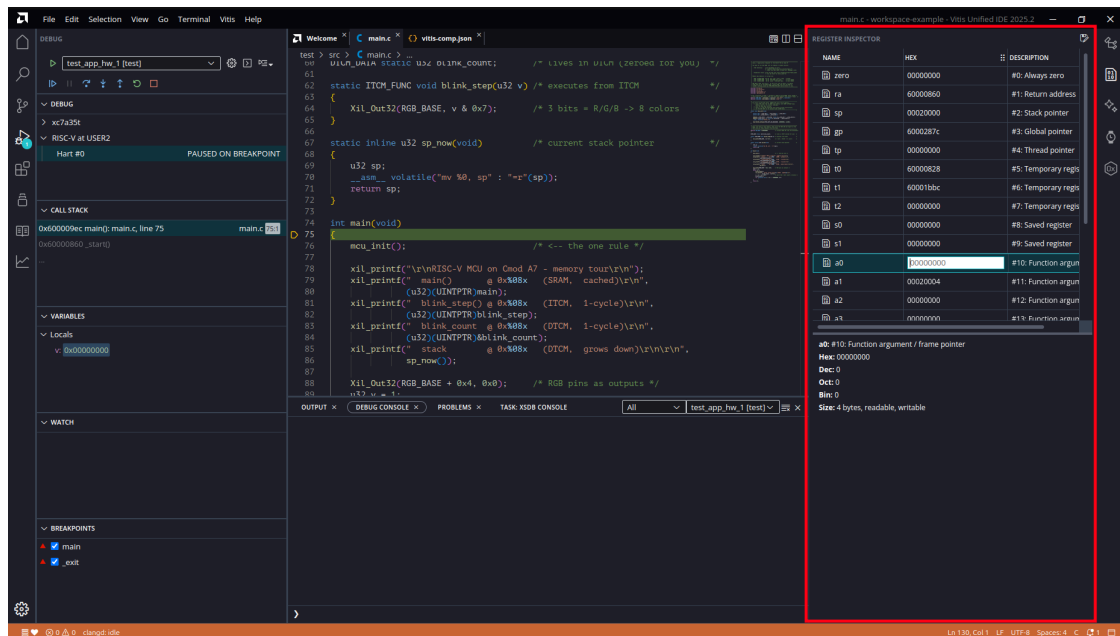


Figure 38: Register Inspector

### 7.9.3 Disassembly View

The Disassembly view shows the executing machine code with source interleaving. The instruction addresses around `main()` all fall in `0x6000_0xxx`; the program runs from SRAM.

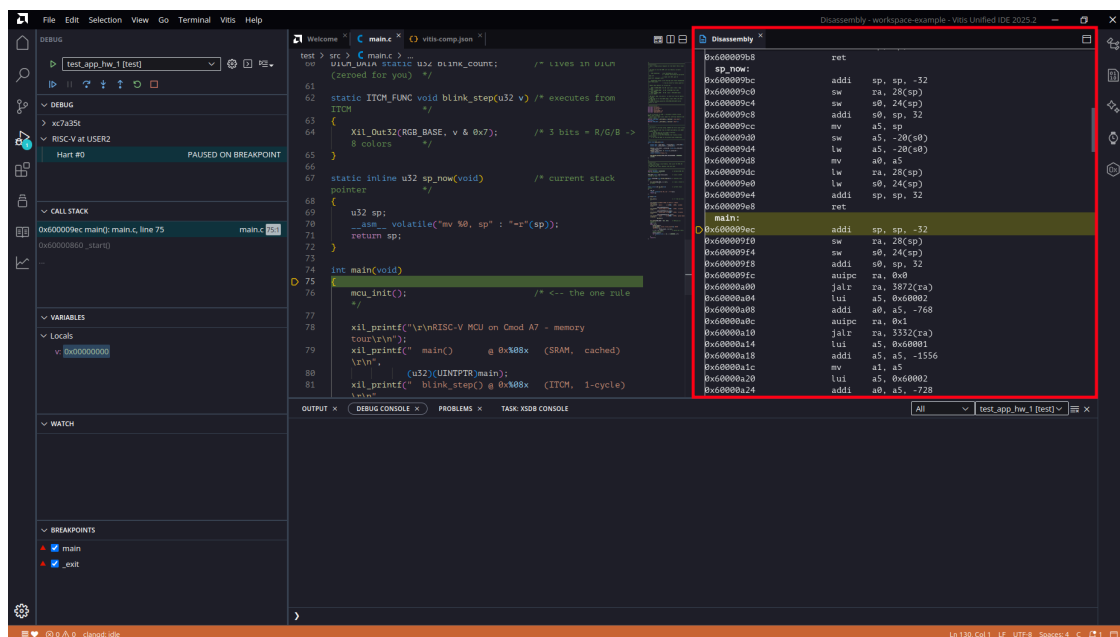


Figure 39: Disassembly

## 7.10 Troubleshooting

**Platform build fails with error: passing argument 2 of 'strncpy' ... [-Wint-conversion] in xclk\_wiz.c (reported as "Error in generating platform").** Two conditions combine to cause this error:

- The clock-management driver source that ships with Vitis 2025.2 (`clk_wiz v1_10`) contains a misplaced parenthesis in `clk_wiz.c` line 839.
- If another RISC-V toolchain with GCC 14 or newer is on your PATH (for example one installed at `/opt/riscv` for assembly coursework), the BSP build may select it instead of the Vitis-bundled compiler. GCC 14+ treats this construct as an error; the bundled compiler treats it as a warning.

The offending line as displayed in the IDE:

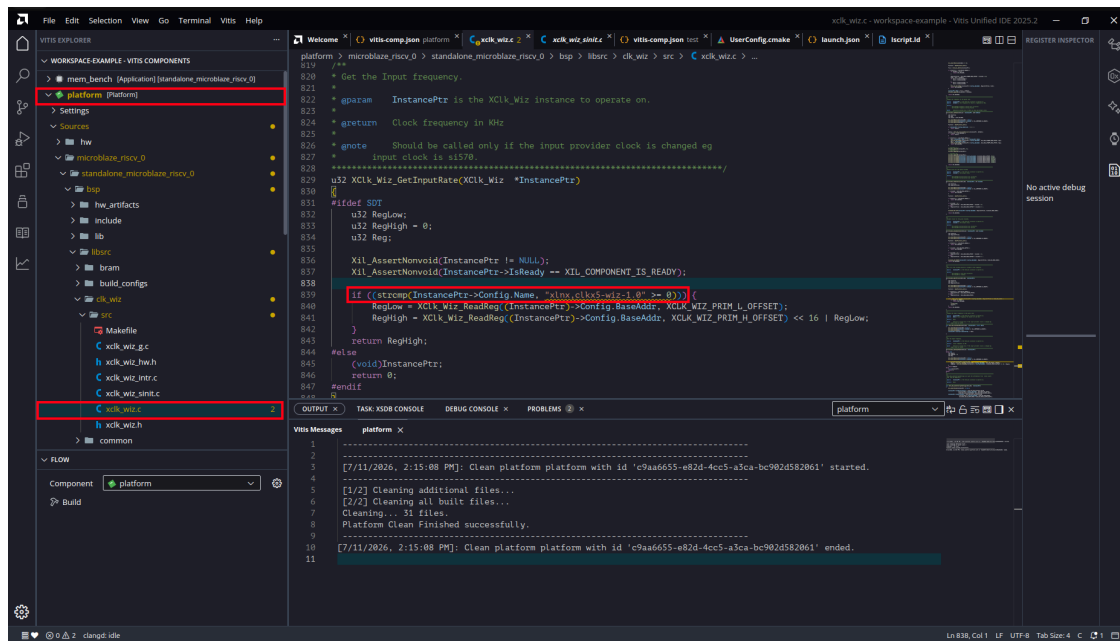


Figure 40: clk\_wiz Driver Bug

Fix the source (this fix works with either compiler): change line 839 from

```
strcmp(InstancePtr->Config.Name, "xlnx,clk5-wiz-1.0" >= 0)
```

to

```
strcmp(InstancePtr->Config.Name, "xlnx,clk5-wiz-1.0") >= 0
```

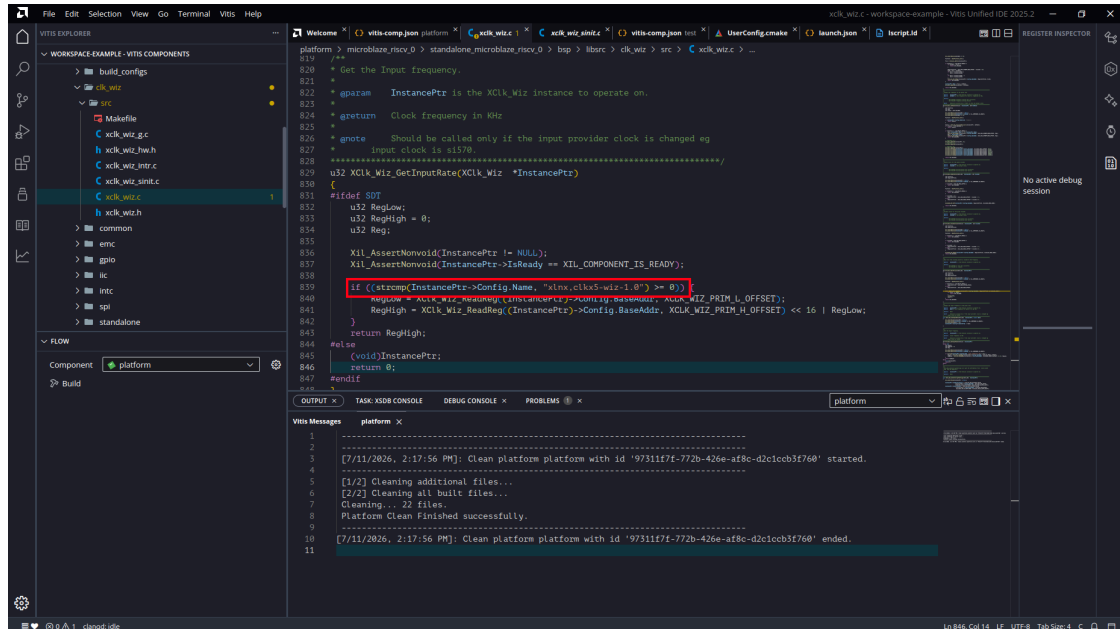


Figure 41: After the Fix

Then rebuild the platform. Apply the same one-line fix to the Vitis installation copy at <install>/2025.2/data/embeddedsw/XilinxProcessorIPLib/drivers/clk\_wiz\_v1\_10/src/xclk\_wiz.c, where <install> is the Xilinx installation directory (/opt/Xilinx by default on Linux). The platform re-imports driver sources whenever the BSP is regenerated, so an unpatched installation reintroduces the error.

To check which compiler configured the BSP, read platform/.../bsp/libsrc/build\_configs/gen\_bsp/compile\_commands.json. To force reselection, delete platform/.../bsp/libsrc/build\_configs and rebuild from a session whose PATH does not contain the other toolchain.

**The application runs (RGB LED cycles) but the USB console is silent.** `xil_printf` output is directed to the DIP-header UART. The BSP stdout has reverted to `uart_1`: redo Section 7.6, then rebuild both the platform and the application. The compiled setting is recorded in `platform/export/platform/sw/standalone_microblaze_riscv_0/include/bspconfig.h`: `STDOUT_BASEADDRESS` must read `0x40300000` (`uart_USB`), not `0x40310000` (`uart_1`).

## 8 Standalone Boot

This chapter describes how an application is stored in the QSPI flash and started automatically at power-on. A small bootloader (AMD's standard SREC SPI Bootloader, carried inside the hardware image) copies the application from flash into SRAM and starts it, with no PC attached, in under a second after power is applied.

The walkthrough covers the complete path: building the bootloader, merging it into the hardware image, converting an application to a flash image, and programming both into the flash. Everything after the hardware image itself is done in the Vitis Unified IDE.

### 8.1 How Standalone Boot Works

```
Power-on → FPGA configures itself from QSPI flash (<1 s)
          → Bootloader (in Block RAM, part of the hardware image) starts
          → Reads the application image (SREC) from flash @ 0x220000
          → Copies it to the addresses the image names (SRAM)
          → Jumps to the application's entry point
```

The flash holds two independent regions, read by two different readers:

| Flash offset       | Contents                                                                        | Read by                                    |
|--------------------|---------------------------------------------------------------------------------|--------------------------------------------|
| 0x000000–0x21FFFF  | Hardware image (bitstream, with the bootloader in its Block RAM initialization) | The FPGA configuration engine, at power-on |
| 0x220000–0x3FFFFFF | Application slot (SREC image, 1.875 MB)                                         | The bootloader                             |

Block RAM is volatile, so the bootloader cannot live there across a power cycle. Its persistent copy travels inside the bitstream: a bitstream carries the initial contents of every block RAM, and the CPU's local memory is built from block RAM. At every power-on the configuration engine reloads the bitstream from flash, which restores the bootloader. Like a mask ROM, it cannot be corrupted from software. The CPU then leaves reset at address 0x0 and executes it.

The application is stored as an SREC image (Motorola S-record), a text format in which every line carries its own destination address. The bootloader copies each record to the address it names, so the linker script decides where the program runs. The course template links to SRAM at 0x6000\_0000, so the image is loaded there.

| Memory            | Size   | Role                                                                     |
|-------------------|--------|--------------------------------------------------------------------------|
| QSPI flash        | 4 MB   | hardware image + bootloader · application slot @ 0x220000 (non-volatile) |
| SRAM @ 0x60000000 | 512 KB | where the application executes (code/data/heap, I/D-cached)              |
| BRAM @ 0x00000000 | 128 KB | 32 KB bootloader · 32 KB ITCM @ 0x8000 · 64 KB DTCM @ 0x10000            |

**Note:** The repository ships the results of this walkthrough prebuilt in `release/:boot_srec.bit` is the hardware image with the bootloader already merged in. To deploy an application with the stock bootloader, skip directly to Section 8.7.

### 8.2 Prerequisites

- A workspace with the platform and a built application, prepared as described in Section 7
- The hardware image and its memory map from the repository: `release/top.bit` and `release/top_wrapper.mmi`
- Cmod A7-35T board connected via its micro-USB cable
- A serial terminal program for the board's console output

### 8.3 Create the Bootloader Component

The bootloader is not custom code: it is the stock **SREC SPI Bootloader** example that ships with Vitis, built like any other application component.

#### 8.3.1 New Application from Template

Open the **Examples** view from the left activity bar. Under *Embedded Software Examples*, select **SREC SPI Bootloader** and click **Create Application Component from Template**.

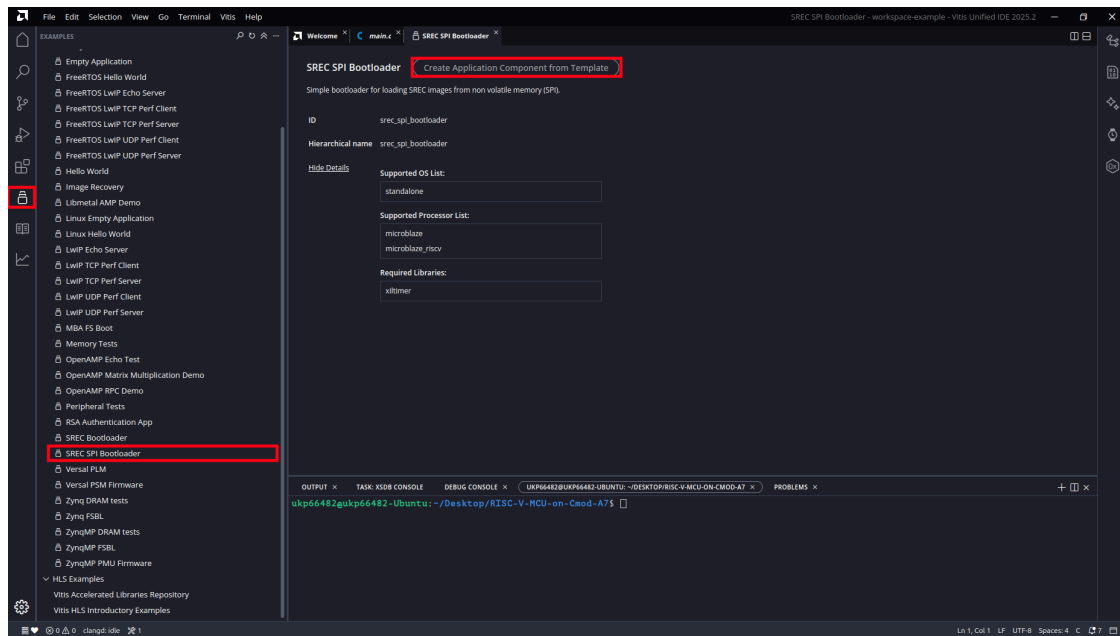


Figure 42: SREC SPI Bootloader Template

### 8.3.2 Name and Location

Keep the proposed name `srec_spi_bootloader` and the workspace directory as the location. Click **Next**.

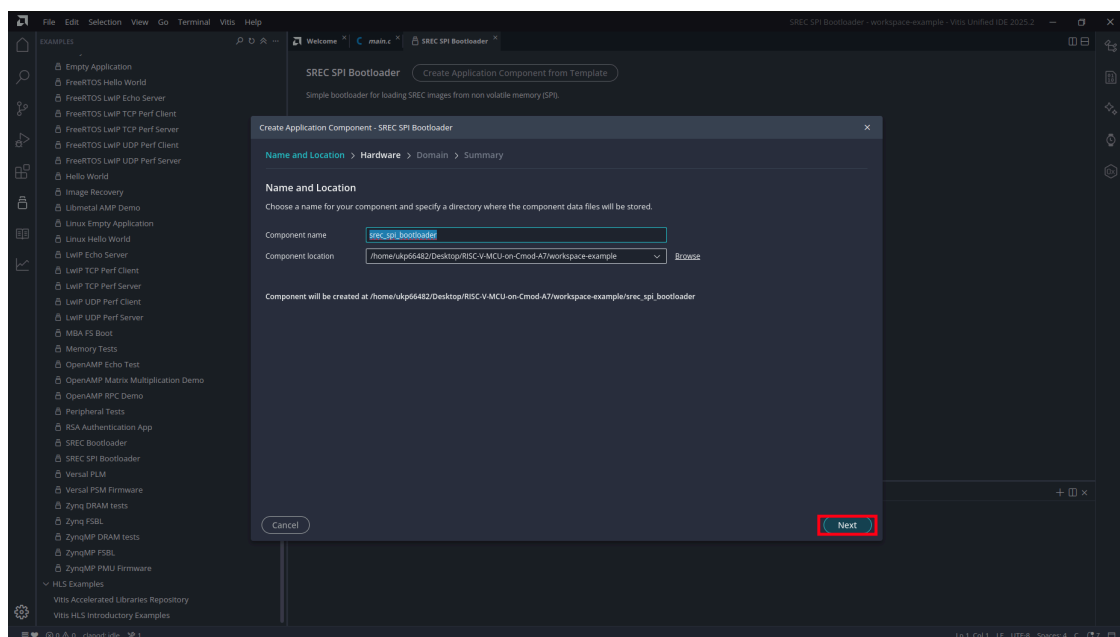


Figure 43: Bootloader Name and Location

### 8.3.3 Select the Platform

Select the platform component (Board: `cmod_a7-35t`). Click **Next**.

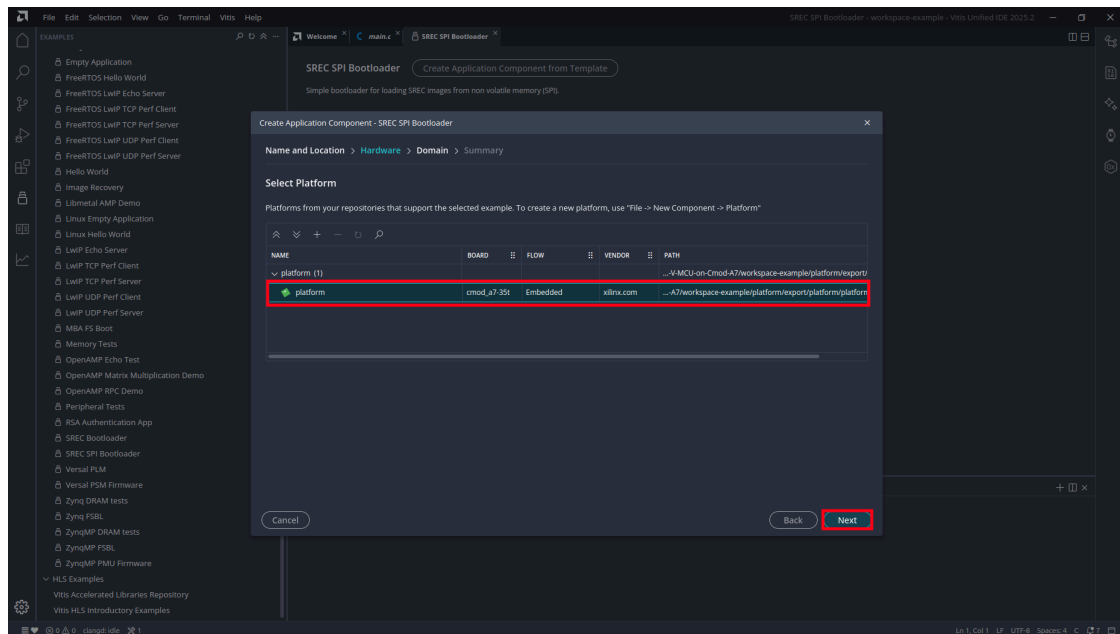


Figure 44: Select Platform

### 8.3.4 Summary

Review the settings (the domain is standalone\_microblaze\_riscv\_0) and click **Finish**.

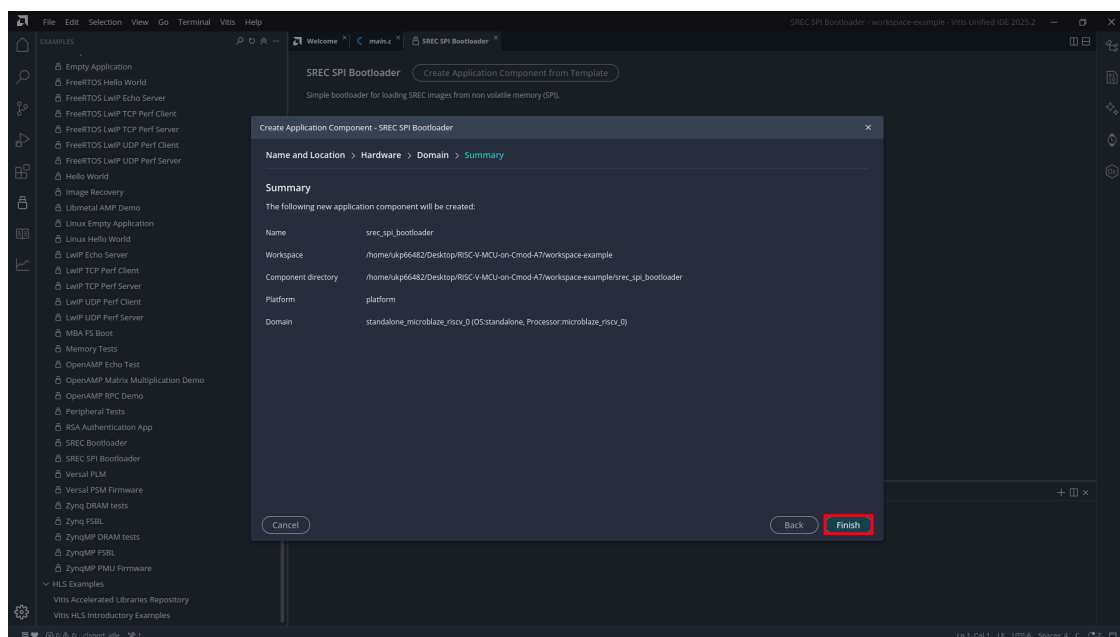


Figure 45: Bootloader Summary

**Note:** The template automatically selects this board's QSPI flash controller, and it links the bootloader to Block RAM at 0x0, so the bootloader can copy an application into SRAM without overwriting itself. Neither setting needs to be adjusted.

## 8.4 Configure the Bootloader

Two source edits configure the template for this board. Both files are under the component's `src/` folder.

### 8.4.1 Set the Application Slot Address

`blconfig.h` defines where in the flash the bootloader looks for the application. The template ships with a placeholder address and a `#warning` that reminds you to change it:

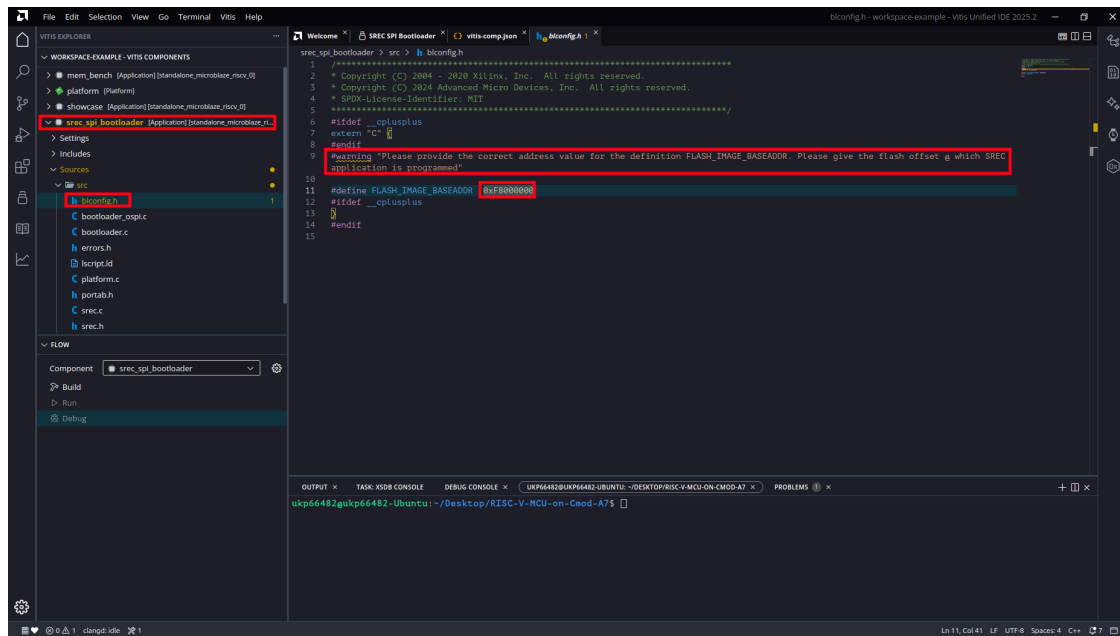


Figure 46: blconfig.h As Generated

Set FLASH\_IMAGE\_BASEADDR to 0x00220000 (the application slot, which starts immediately after the bitstream region) and comment out the #warning line:

```
#define FLASH_IMAGE_BASEADDR 0x00220000
```

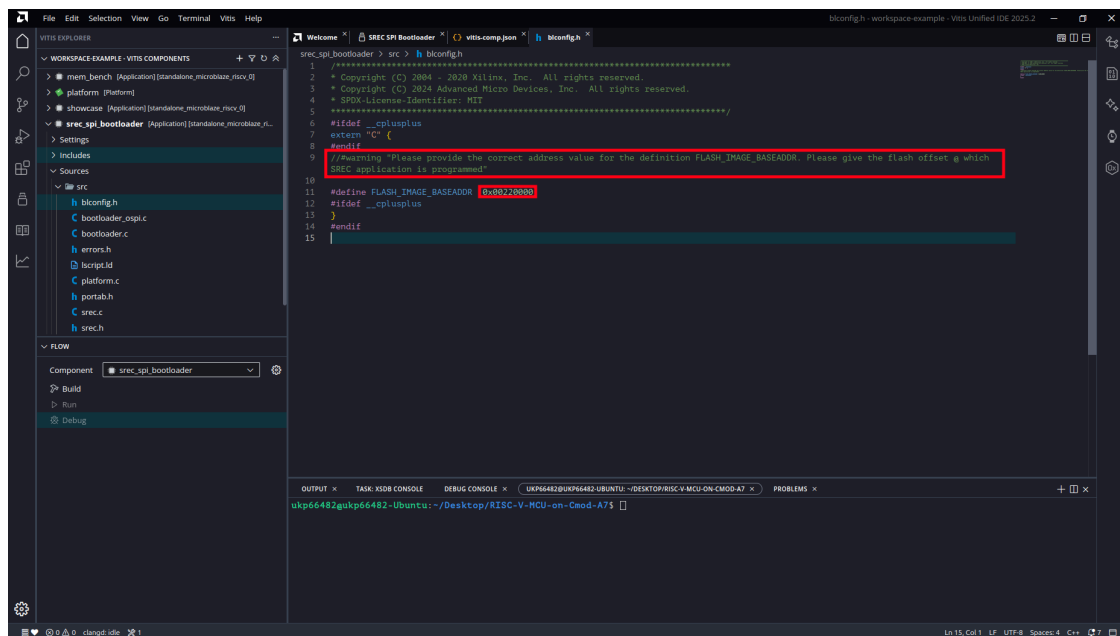


Figure 47: blconfig.h Configured

## 8.4.2 Disable Verbose Output

bootloader.c defines VERBOSE by default (line 45):

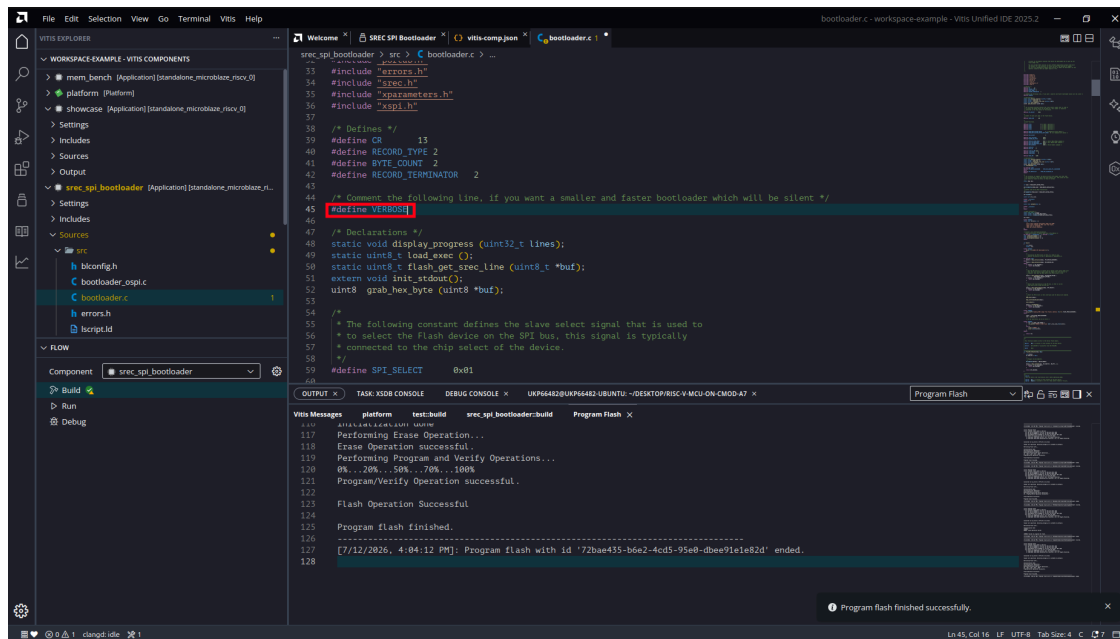


Figure 48: VERBOSE As Generated

Comment it out:

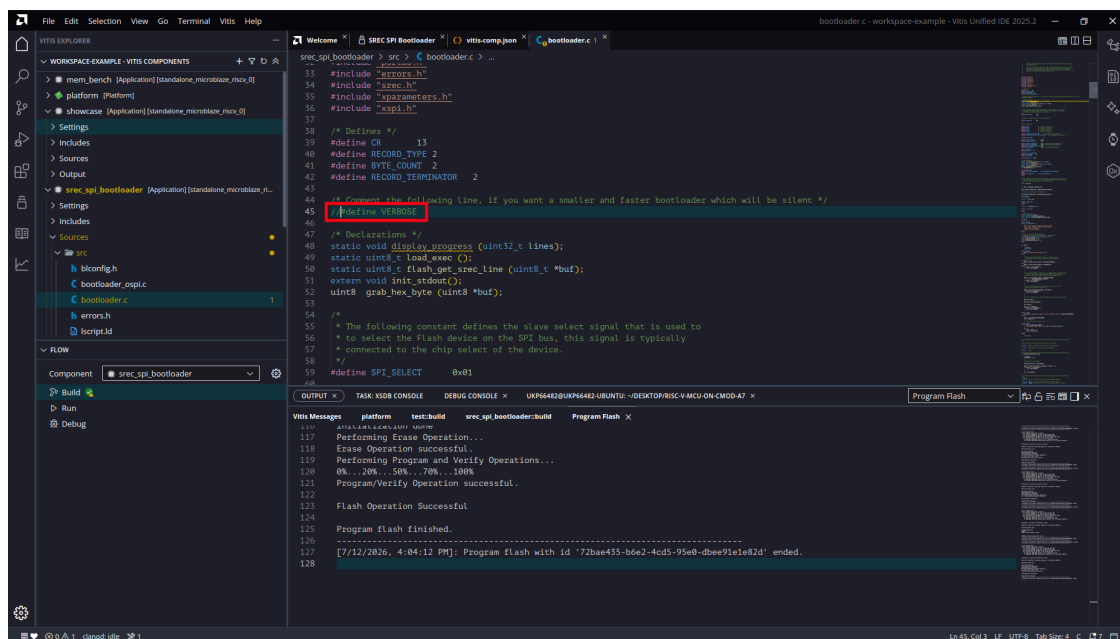


Figure 49: VERBOSE Disabled

**Note:** With VERBOSE enabled the bootloader prints progress for every record it copies, through a UART whose baud divisor it never initializes. The application still boots, but loading takes many seconds instead of well under one second. Disabling VERBOSE gives a silent bootloader and restores the sub-second boot. The definition returns whenever the template is regenerated, so re-check this line when creating a new bootloader component.

## 8.5 Build the Bootloader

In the **FLOW** panel, set the component to `srec_spi_bootloader` and click **Build**.

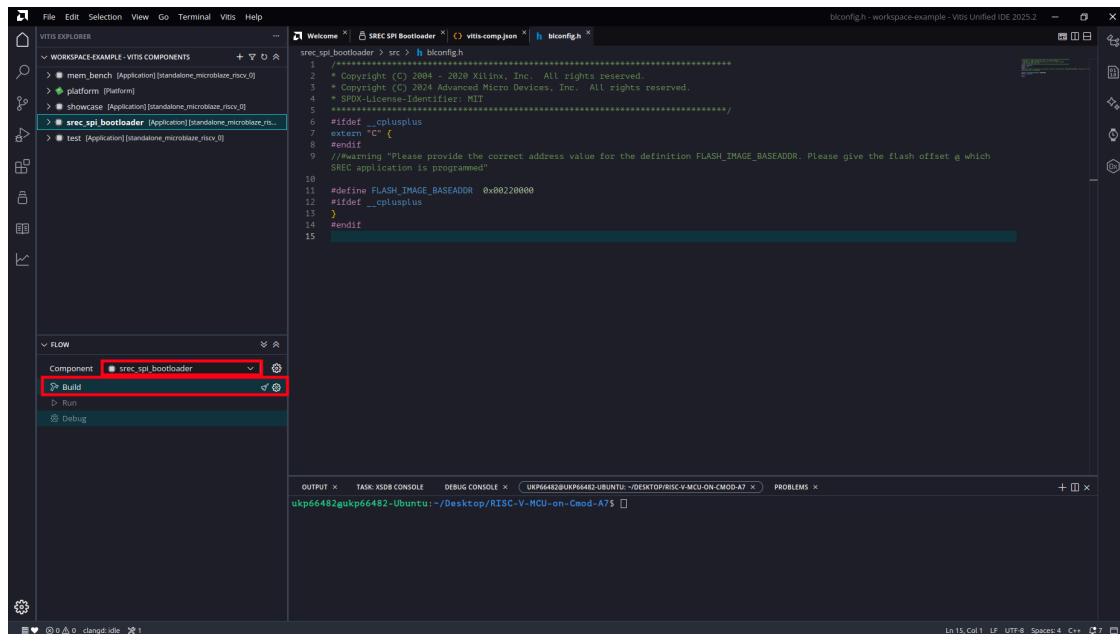


Figure 50: Build the Bootloader

The build produces `srec_spi_bootloader.elf` under the component's Output tree (about 16 KB of code, well inside its 32 KB Block RAM region).

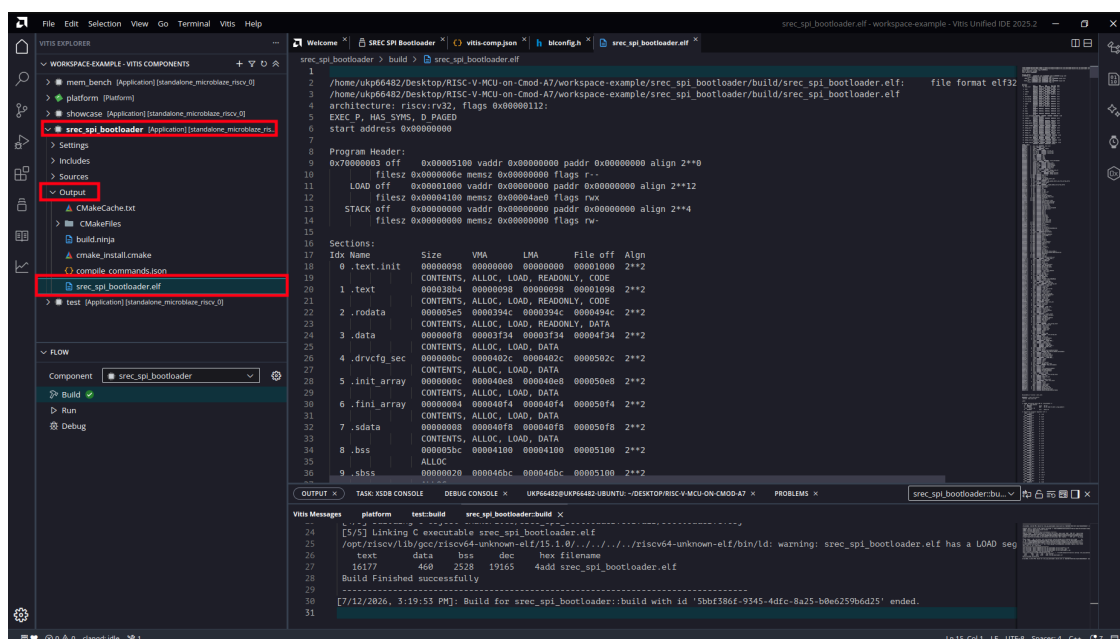


Figure 51: Bootloader ELF Built

## 8.6 Merge the Bootloader into the Hardware Image

The bootloader ELF is merged into the bitstream's Block RAM initialization with `updatemem`, a command-line tool that ships with Vitis. Only the RAM initialization words change; the logic, routing, and timing of the hardware image are untouched, so the merge takes seconds and needs no hardware rebuild.

Open a terminal in the repository root (**Terminal > New Terminal**) and run:

```
updatemem -force \
-meminfo release/top_wrapper.mmi \
-data workspace-example/srec_spi_bootloader/build/srec_spi_bootloader.elf \
-bit release/top.bit \
-proc top_i/microblaze_riscv_0 \
-out release/boot_srec.bit
```

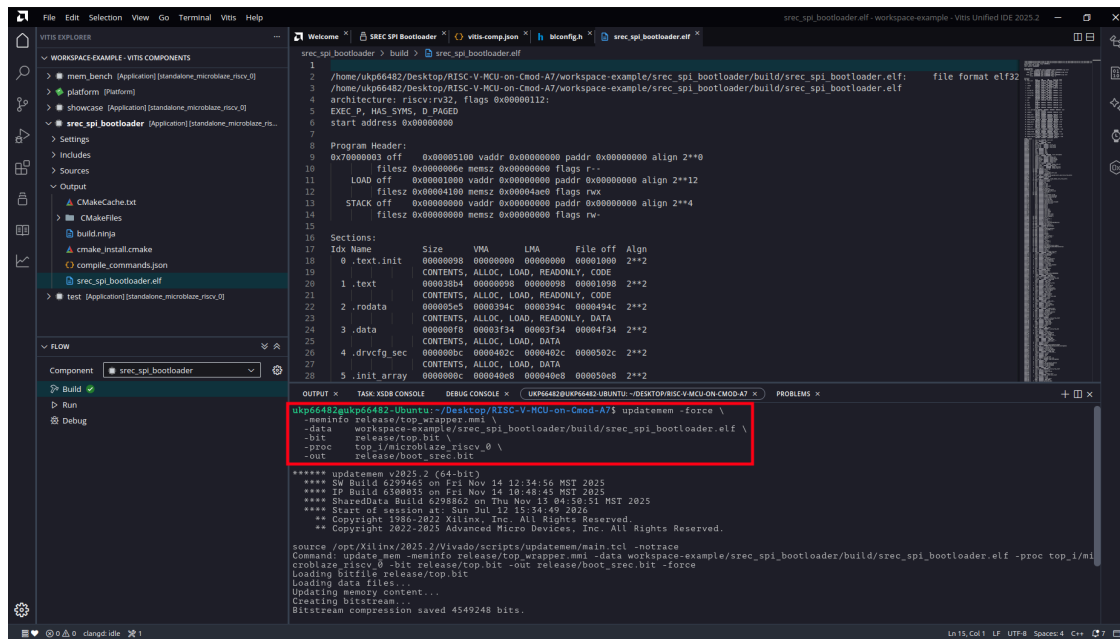


Figure 52: updatemem in the Integrated Terminal

The inputs and what they mean:

| Argument | File                     | Purpose                                                                             |
|----------|--------------------------|-------------------------------------------------------------------------------------|
| -meminfo | top_wrapper.mmi          | Which physical Block RAM cell holds which CPU address (fixed at hardware placement) |
| -data    | srec_spi_bootloader.elf  | The program to place into Block RAM                                                 |
| -bit     | top.bit                  | The hardware image, Block RAM blank                                                 |
| -proc    | top_i/microblaze_riscv_0 | Which processor's memory to initialize                                              |
| -out     | boot_srec.bit            | The bootable hardware image                                                         |

**Note:** A successful run prints updatemem completed successfully and rewrites the output file. If the tool prints only its usage text, one of the paths is wrong and no output is produced; check the output file's timestamp before using it. The MMI file must come from the same hardware build as the bitstream.

## 8.7 Prepare the Application Image

The bootloader reads SREC text, so the application ELF is converted with the toolchain's objcopy. This walkthrough deploys the test application built in Section 7; the same ELF runs unchanged under JTAG and from flash.

```
riscv32-amd-linux-gnu-objcopy -O srec \
workspace-example/test/build/test.elf \
workspace-example/test/build/test.srec
```

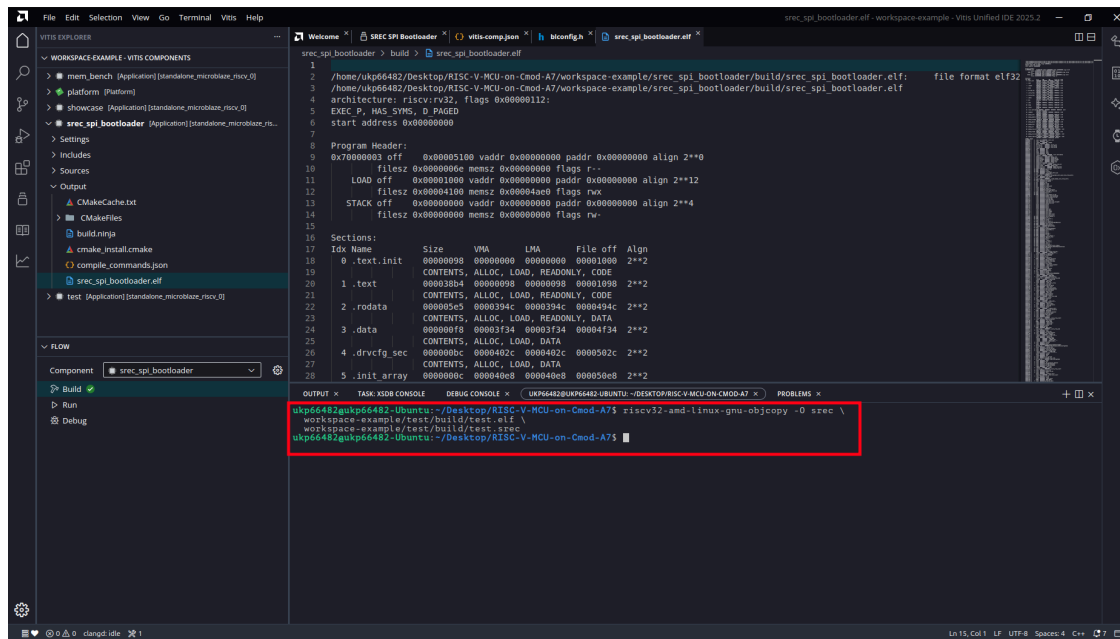


Figure 53: objcopy to SREC

Then give the image a .bin extension:

```
cp workspace-example/test/build/test.srec \
workspace-example/test/build/test_srec.bin
```

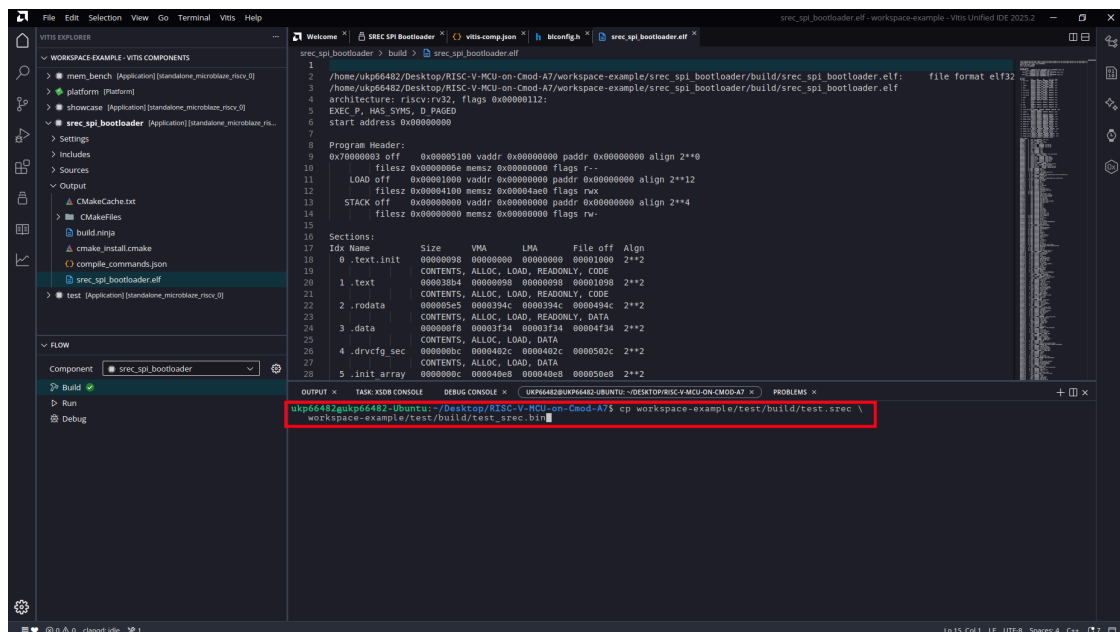


Figure 54: Rename to .bin

**Note:** The extension matters. The flash programmer treats a .srec file as an image format and decodes it to the memory addresses inside it; a .bin file is written to the flash byte for byte, which is what the bootloader expects to find in the slot. The file name itself is free; only the .bin extension is significant.

Each S3 line of the SREC text embeds its destination address; with the course linker script every record falls in SRAM (addresses beginning 6000). The image is about three times the size of the binary it encodes; the 1.875 MB slot therefore never limits an application before the 512 KB SRAM does.

## 8.8 Program the Flash

Both flash regions are written from **Vitis > Program Flash...** in the menu bar. Each write erases and programs only the address range of its own image, so the two regions never disturb each other.

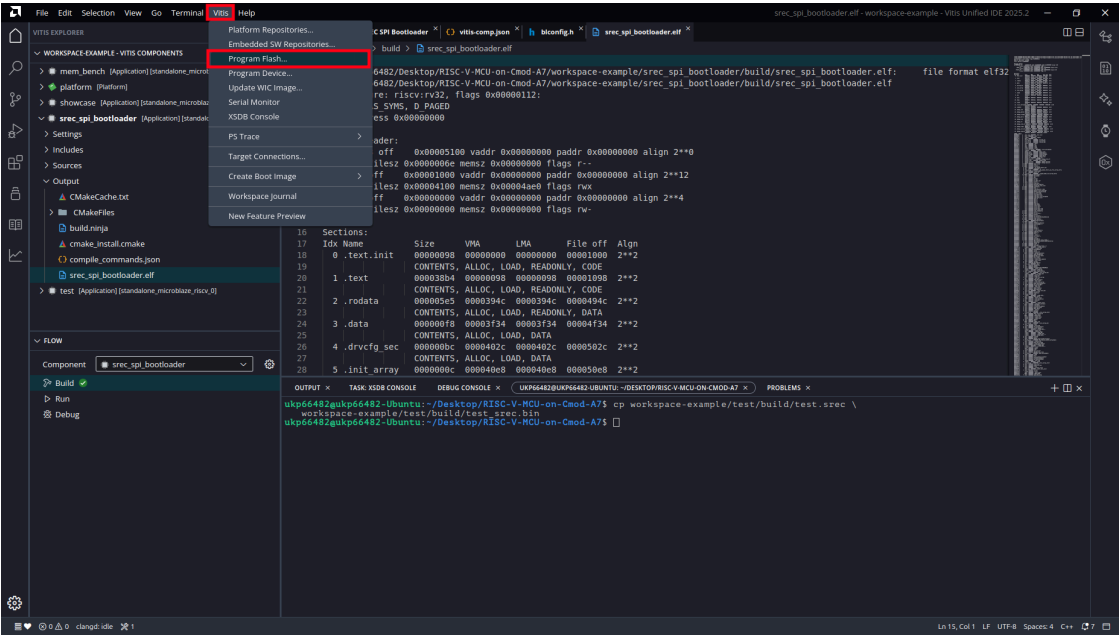


Figure 55: Vitis Program Flash Menu

8.8.1 Program the Hardware Image

This step is needed once, and again only after the bootloader or the hardware design changes. In the *Program Flash Memory* dialog set:

| Field              | Value                   |
|--------------------|-------------------------|
| Image File         | release/boot_srec.bit   |
| Offset             | 0x0                     |
| Flash Type         | mx25l3273f-spi-x1_x2_x4 |
| Verify after flash | checked                 |

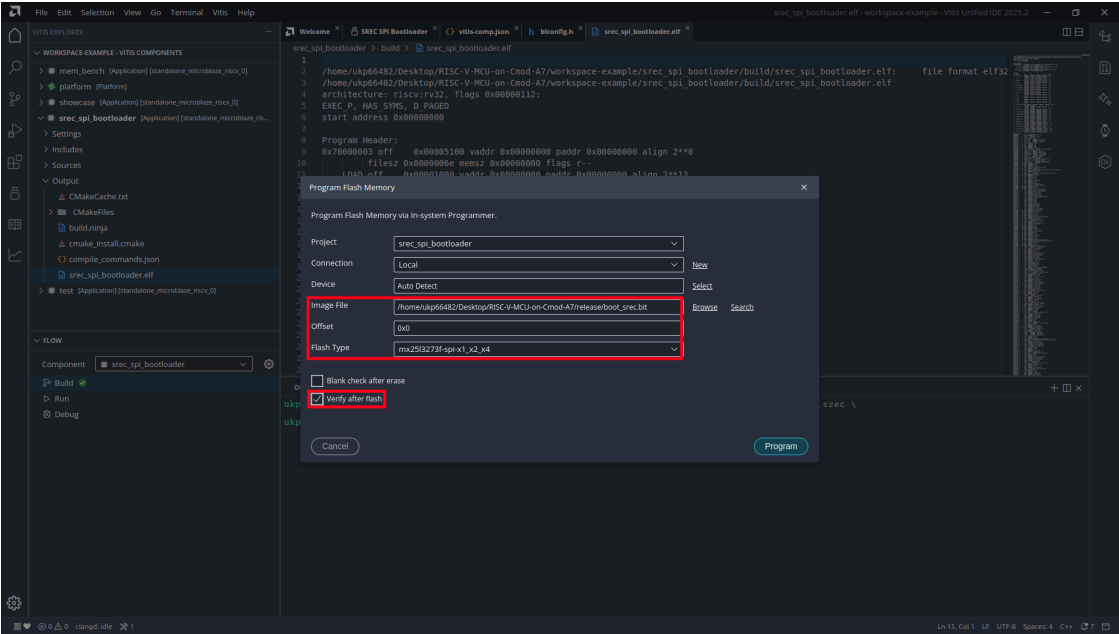


Figure 56: Program the Hardware Image

Click **Program** and wait for *Program flash* finished.

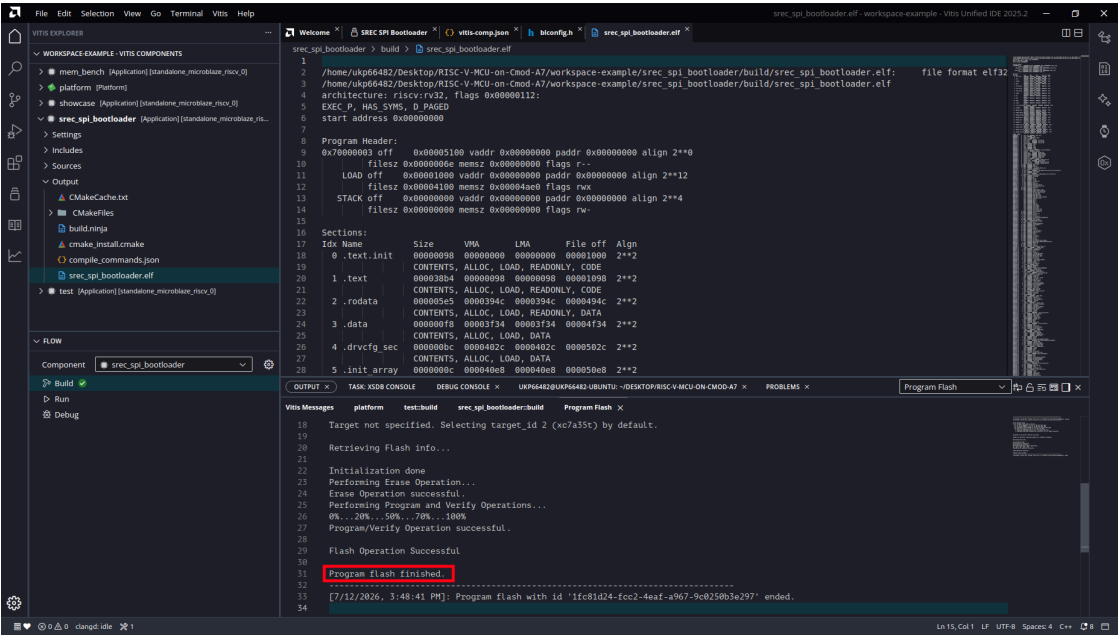


Figure 57: Hardware Image Programmed

**Note:** The flash type entry covers the board's Macronix MX25L3233F device. Older boards carry a Micron part instead; if the IC3 marking says N25Q032, select n25q32-3.3v-spi-x1\_x2\_x4.

8.8.2 Program the Application

Open Vitis > Program Flash... again and set:

| Field              | Value                      |
|--------------------|----------------------------|
| Image File         | the .bin application image |
| Offset             | 0x220000                   |
| Flash Type         | mx25l3273f-spi-x1_x2_x4    |
| Verify after flash | checked                    |

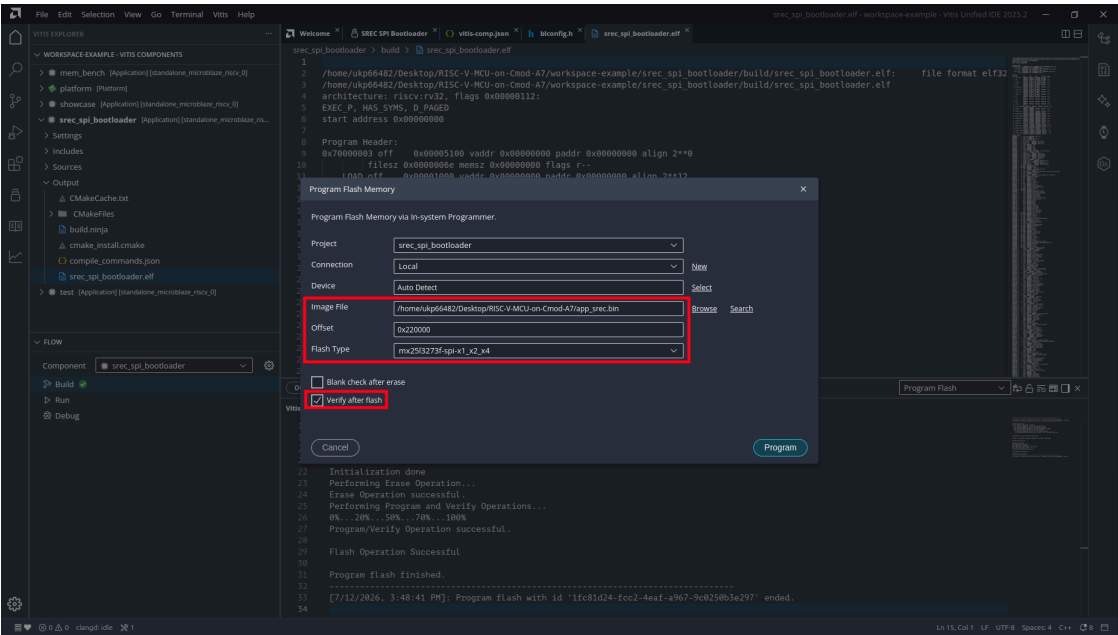


Figure 58: Program the Application Slot

Click **Program** and wait for *Program flash finished*.

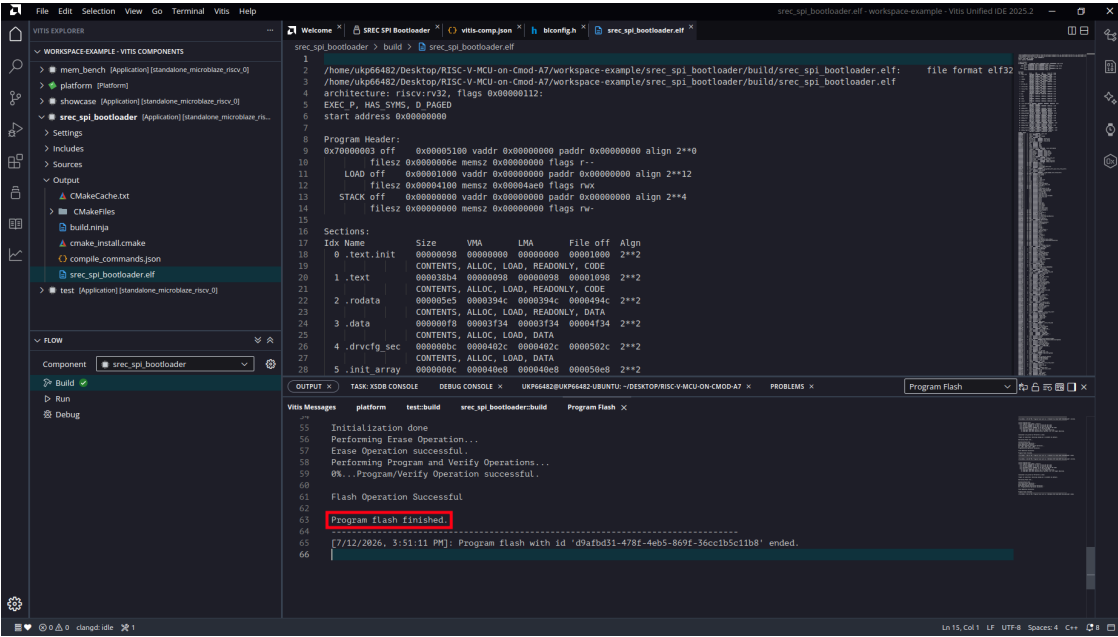


Figure 59: Application Slot Programmed

8.9 Boot and Verify

Disconnect the USB cable and plug it back in. This is a true cold boot: the FPGA configures itself from the flash, the bootloader loads the application, and the application runs, with no debugger and no IDE involved.

Open a serial terminal at 115200 8N1 (the Vitis **Serial Monitor**, GTKTerm, or pyserial's miniterm; the board enumerates as two ports and the UART is usually the higher-numbered one). The course template prints its memory tour, followed by the running status output:

```
RISC-V MCU on Cmod A7 - memory tour
main()      @ 0x600009EC (SRAM,  cached)
blink_step() @ 0x00008000 (ITCM,  1-cycle)
blink_count @ 0x00010000 (DTCM,  1-cycle)
stack       @ 0x0001FFC0 (DTCM,  grows down)
```

**Note:** The reset button (BTN0) restarts the CPU into the bootloader, which re-copies the application from flash, so a reset behaves like a power-cycle. Modified .data globals are restored because the image is re-read from flash at every boot.

8.10 Updating the Application

After the one-time setup above, the day-to-day deployment loop has three steps, none of which touch the bootloader or the hardware image:

- 1. Build the application in Vitis.
- 2. Convert and rename the image (the two commands in the application image step).
- 3. **Vitis > Program Flash...** to offset 0x220000.

Power-cycle, and the new application boots.

8.11 Working with JTAG Debug Mode

Both modes coexist on the same board with no switching:

|                        | JTAG (Vitis Run/Debug)        | Standalone (flash)             |
|------------------------|-------------------------------|--------------------------------|
| Code goes to           | RAM directly (volatile)       | Flash (persistent)             |
| After power-cycle      | the flashed application boots | the deployed application boots |
| Breakpoints / stepping | yes                           | no                             |
| Typical use            | daily development loop        | shipping a finished program    |

The typical development cycle is edit, build, and JTAG Run/Debug. When the program works, deploy it to flash once. JTAG always takes priority over the bootloader: it halts the CPU before the bootloader runs, so the flash contents cannot interfere with a debug session.

## 8.12 Troubleshooting

1. **Boot works but takes many seconds:** `VERBOSE` is still defined in `bootloader.c`. Comment it out, rebuild the bootloader, re-run `updatemem`, and reprogram the hardware image.
2. **Nothing at all after power-on (no LED, no output):** The flash holds `top.bit` (blank Block RAM, no bootloader) instead of `boot_srec.bit`, or the hardware image region is empty or corrupt. Reprogram the hardware image. JTAG remains available for recovery.
3. **updatemem printed only usage text:** One of the argument paths is wrong; no output file was written, and reprogramming would flash the previous image. Fix the path and confirm the `updatemem completed successfully` message.
4. **Program Flash fails with Flash Operation Failed:** Another process is holding the JTAG connection, typically a Vitis debug session or an XSDB console. Close it (or unplug and replug the board) and retry.
5. **Garbage on the terminal:** The baud rate must be 115200. The course template sets the UART divisor in `mcu_init()`; the plain Hello World template does not set it at all.
6. **Deployed, but silent after power-on (works under JTAG):** The ELF was not linked with the course template's `lscript.ld`. The bootloader honors the SREC record addresses, and a default all-BRAM layout collides with the bootloader itself. Rebuild with the template and redeploy.
7. **Board boots an old application:** The flash write did not complete. Repeat the application programming step and confirm the verify pass succeeds.